

METACASTER: 面向轻量级时间序列预测器的端到端小样本学习的元工具优化智能体

ChengAo Shen¹, Wenchao Yu², Fangyu Wu³, Dongjin Song⁴, Hanghang Tong⁵,
Dongsheng Luo⁶, Wei Cheng², Haifeng Chen², Jingchao Ni¹

¹University of Houston, ²NEC Labs, ³University of Waterloo, ⁴University of Connecticut,

⁵University of Illinois at Urbana-Champaign, ⁶Singapore Management University

¹{cshen9, jni7}@uh.edu, ²{wyu, weicheng, haifeng}@nec-labs.com,

³fangyu.wu@uwaterloo.ca, ⁴dongjin.song@uconn.edu,

⁵htong@illinois.edu, ⁶dsluo@smu.edu.sg

摘要

时间序列预测 (TSF) 正朝着多模态和智能化方向发展, 然而在资源受限的场景中, 使用基础模型仍然不经济, 此时紧凑、专用的预测器更为可取。然而, 轻量级预测器通常需要大量训练数据, 这限制了它们在数据稀缺、积累缓慢或对隐私敏感的时间序列领域中的应用。为了解决这一困境, 我们研究了轻量级预测器的小样本学习这一具有挑战性的问题。我们提出了 METACASTER, 一个元工具优化的多智能体框架, 它利用智能体数据生成, 仅从少量示例和文本上下文中自动训练专用的轻量级预测器。我们的工作凸显了一种新的 TSF 范式, 其中智能体不是作为预测器, 而是作为中间工程师, 为部署准备高效、任务特定的预测器。在 18 个数据集、23 个最先进的轻量级预测器和 14 个基线上的实验表明, METACASTER 在保持高质量 TSF 性能的同时, 实现了数据效率和计算效率。¹

1 引言

时间序列预测 (TSF) 支撑着地球科学、医疗保健和能源等领域的决策和智能 (Koprinska 等, 2018; Morid 等, 2023; Ardid 等, 2025)。与专注于未来时间步数值预测的传统 TSF 不同, 新兴的 TSF 设置是多模态的, 融合了描述领域、事件或外部条件的文本上下文 (Jiang 等, 2025)。这一转变是由大语言模型 (LLMs) 和人工智能智能体的进步所驱动的。如图 1 所示, 近期基于 LLM 的 TSF 方法主要有两种范式: (a) LLM 作为预测器——其中预训练的 LLM 充当

预测骨干网络配备额外的跨模态适配器和时间序列预测头 (例如, TimeLLM (Jin 等, 2024)、S2IPLLM (Pan 等, 2024)、TimeVLM (Zhong 等, 2025) 等); 以及 (b) 智能体即预测器——其中大语言模型智能体在上下文提示中解读时间序列, 推理未来趋势, 并以对话或可视化方式生成预测 (例如, LLMTime (Gruver 等, 2023)、TimeOmni-1 (Guan 等, 2026a)、Nexus (Das 等, 2026) 等)。尽管取得了令人鼓舞的进展, 但将大语言模型用作预测的计算核心在很大程度上受到离散语言标记与连续时间序列数值之间模态差距的阻碍。因此, 与专门的非大语言模型预测器相比, 图 1 (a) 和 (b) 中范式的有效性仍存在争议 (Tan 等, 2024; Merrill 等, 2024)。基准研究一致表明, 专用数值模型 (如 DLinear (Zeng 等, 2023) 和 PatchTST (Nie 等, 2023)) 可以优于基于大语言模型的方法 (Shen 等, 2025a), 这推动了时间序列基础模型 (TSFM) 的兴起, 例如 Moirai (Woo 等, 2024)、Chronos (Ansari 等, 2024) 和 Sundial (Liu 等, 2025c)。然而, 使用大型模型——无论是大语言模型还是时间序列基础模型——作为一刀切的解决方案引发了人们对可持续性和碳足迹的担忧 (Bolón-Canedo 等, 2024)。因此, 在资源受限环境 (例如边缘设备和小型组织) 中快速部署的需求引发了人们对轻量级预测器的兴趣日益增长 (Ma 等, 2024), 类似于小语言模型 (SLM) 的兴起 (Belcak 等, 2025)。值得注意的是, 最近的研究表明, 轻量级预测器在单个时间序列预测任务中可以与大语言模型和时间序列基础模型相媲美 (Shen 等, 2025b), 其主要局限性在于需要足够的下游训练数据, 因为它们缺乏大规模预训练。在实践中, 收集用于训练的长历史时间序列可能会显著延迟部署。在医疗保健等隐私敏感领域以及

¹METACASTER 的代码可在 <https://github.com/D2I-Group/metacaster> 获取。

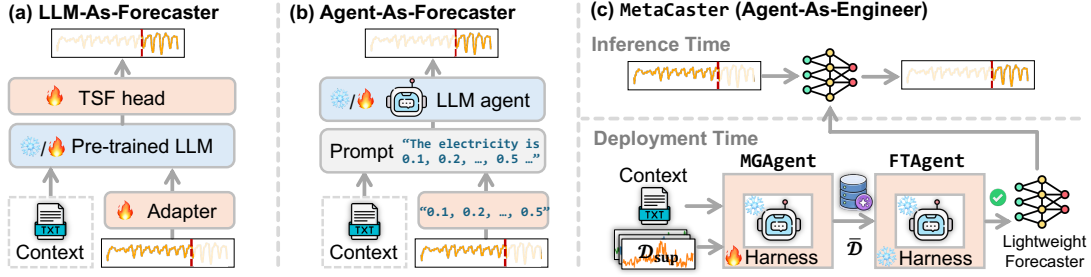


图 1: 使用大语言模型进行时间序列预测的不同范式比较。METACASTER 是所提出的模型。

金融领域，获取大型训练数据集甚至可能不可行。这提出了一个具有挑战性的问题：轻量级预测器能否仅用少量样本进行有效训练？尽管此类模型在没有大规模预训练的情况下极易过拟合，但我们证明在智能体人工智能时代解决方案是可能的。

为应对这一挑战，本文提出元铸造者（METACASTER），一种面向轻量级预测器端到端小样本学习的元框架优化智能体。如图 1(c) 所示，元铸造者的智能体并非直接生成预测，而是充当中间角色，为目标时间序列预测任务准备专用预测器。在部署阶段，给定小样本支持集 $\mathcal{D}_{\text{sup}}$ 与上下文描述 C ，元铸造者调用两个智能体：（1）元生成智能体参考 $\mathcal{D}_{\text{sup}}$ 与 C ，生成符合领域约束的充足数据集 $\bar{\mathcal{D}}$ ；（2）微调智能体利用 $\bar{\mathcal{D}}$ 训练并筛选最优轻量级预测器。为支持微调智能体，本文将 23 种最先进的轻量级预测器（2022-2026 年）整合为统一应用程序编程接口的轻量级时序库（§ 3.5）。与基础模型微调类似，元铸造者在部署阶段执行任务适配，但通过新颖的智能体与大语言模型应用程序编程接口调用实现更高效率。部署完成后，仅使用选定的轻量级预测器进行推理。

与现有专注于数据仿真的时间序列生成模型不同（Huang 等，2025b；Gu 等，2025；Ge 等，2025），元铸造者生成时间序列以专门提升预测性能。据本文所知，这是首个将数据生成与预测器质量对齐的框架。受近期关于智能体框架（Harness）重要性研究的启发（Li 等，2026a；Lee 等，2026）——即大语言模型周围的基础设施（如系统提示、技能、工具）——本文利用元框架智能体优化元生成智能体的框架（图 2），其效率高于微调大语言模型。优化后的框架还具有

跨不同大语言模型的可迁移性，支持下游部署中灵活的应用程序编程接口切换，如 § 4.2 所示。综上，本文贡献如下：• 研究轻量级预测器小样本学习的挑战性问题；• 提出元铸造者，一种新颖的元框架优化多智能体框架，在时间序列预测任务中实现大模型级性能与高效推理；• 将 20 余种最先进的轻量级预测器整合为轻量级时序库，该统一库将随元铸造者一同发布；• 在 18 个数据集上与 14 个基线进行综合实验，验证元铸造者的有效性。

2 相关工作

基于大语言模型的时间序列预测。如第 1 节所述，许多现有基于大语言模型的时间序列预测模型采用大语言模型作为预测器

（LLM-As-Forecaster）（Zhou 等，2023；Jin 等，2024；Pan 等，2024；Liu 等，2024，2025a；Wang 等，2025；Zhong 等，2025）或智能体作为预测器（Agent-As-Forecaster）（Xue 和 Salim，2023；Gruver 等，2023；Das 等，2026）。此外，一些多任务时间序列问答智能体可以通过大语言模型推理进行时间序列预测（Kong 等，2025；Guan 等，2026a；Yu 等，2025；Wu 等，2026；Guan 等，2026b）。相比之下，使用工具框架（Harness）进行时间序列预测的智能体作为工程师（Agent-As-Engineer）模型相对较少（Zhao 等，2025；Garza 和 Rosillo Garcia，2025；Zhang 等，2025；Jalori 等，2025）。然而，这些模型不生成时间序列（因此需要大量训练数据），并且从未自动优化其用于时间序列预测的工具框架，这使它们与元铸造者（METACASTER）区别开来。

时间序列生成。我们的工作与时间序列生成模型相关，包括数据增强技术（Luo 等，2023；Yue 等，2022；Iwana 和 Uchida，2021；Wen 等，2021）、关注数据分布的生成模型

(Yoon 等, 2019; Desai 等, 2021; Jeon 等, 2022; Yuan 和 Qiao, 2024), 以及能够编码上下文的基于语言的模型 (Naiman 等, 2024; Huang 等, 2025b; Gu 等, 2025; Ge 等, 2025)。然而, 这些独立的生成器旨在模拟某些数据属性, 而不是直接优化时间序列预测性能, 留下了显著的差距, 正如我们将在第 4 节中展示的那样。

智能体工具框架优化。人工智能智能体正在经历范式转变。最近的发现挑战了仅靠更好的模型就能产生更可靠智能体的假设 (Li 等, 2026a; Ning 等, 2026)。相反, 改进智能体周围的基础设施层, 即智能体工具框架, 可以显著提升其性能 (Li 等, 2026a), 导致对工具框架工程 (Lin 等, 2026) 的关注日益增加, 并出现了在智能体系统内优化文本 (Yuksekgonul 等, 2025) 和工具框架 (Lee 等, 2026) 的新兴技术。与主要依赖监督微调或强化学习的时间序列智能体不同 (Guan 等, 2026a, b; Wu 等, 2026), 据我们所知, 所提出的元铸造者 (METACASTER) 是第一个接受自动工具框架优化的时间序列智能体。

3 所提出的方法

3.1 问题陈述

给定回溯窗口内的多元时间序列 (MTS) $\mathbf{X} = [\mathbf{x}^1, \dots, \mathbf{x}^D]^\top \in \mathbb{R}^{D \times T}$ within a look-back window of length T , where $\mathbf{x} \in \mathbb{R}^T$ ($1 \leq d \leq D$) is a univariate time series (UTS) of d -th variate, the time series forecasting (TSF) 的目标是估计未来 H 个时间步上 MTS 最可能的值, i.e., $\hat{\mathbf{Y}} \in \mathbb{R}^{D \times H}$, 使得估计值与真实值 $\mathbf{Y} = \mathbf{X}_{T+1:T+H} \in \mathbb{R}^{D \times H}$ 之间的差异在诸如均方误差 (MSE) 等度量下最小化。本文集成多个轻量级预测器 $\mathcal{F} = \{f_1, \dots, f_L\}$, 其中 $f_l: \mathbb{R}^{D \times T} \rightarrow \mathbb{R}^{D \times H}$ (1 $\leq l \leq L$) 的是, $\mathcal{F}$ 中 $\mathcal{F}$ 的预测器由于容量较小, 均为非预训练模型。

任务 给定 $\mathcal{F}$, a K 样本支持集 $\mathcal{D}_{\text{sup}} = \{(\mathbf{X}_i, \mathbf{Y}_i)\}_{i=1}^K$, 以及关于目标领域 (如气候学、医疗保健) 的文本上下文描述 C , 任务是在 $\mathcal{F}$ 中训练预测器, 使其在测试集上达到最优预测误差。当 K 较小时, 即少样本场景下, 该任务具有挑战性, 尤其对于 $\mathcal{F}$ 中的小型预测器而言。

3.2 框架概述

为解决该任务, 本文提出 METACASTER, 一个多智能体系统, 基于 $\{\mathcal{D}_{\text{sup}}, C\}$ 中的有限数据生成具有 $N' \gg K$ 的充足数据集 $\bar{\mathcal{D}} = \{(\mathbf{X}_i, \mathbf{Y}_i)\}_{i=1}^{N'}$, 然后将 $\bar{\mathcal{D}}$ 划分为训练集 $\bar{\mathcal{D}}_{\text{tr}}$ 和验证集 $\bar{\mathcal{D}}_{\text{val}}$, 用于训练 $\mathcal{F}$ 中的预测器。与现有注重数据真实性的生成模型不同 (Huang 等, 2025b; Gu 等, 2025; Ge 等, 2025), METACASTER 生成 $\bar{\mathcal{D}}$ 以确保在 $\bar{\mathcal{D}}_{\text{tr}}$ 上训练的预测器性能与在目标领域中相同规模真实数据 (如 $|\bar{\mathcal{D}}_{\text{tr}}|$) 上训练的预测器相当。这一目标差异至关重要: 标准生成器无法保证数据的忠实再现, 由此产生的不匹配可能导致预测器产生偏差。相比之下, METACASTER 直接优化数据生成以提升下游预测性能, 学习生成更适合在目标领域构建有效时间序列预测器的训练数据。为实现智能体优化, 本文采用元框架策略 (Lee 等, 2026), 鉴于近期关于框架对智能体性能具有显著影响的研究发现 (Li 等, 2026a)。图 2 展示了 METACASTER 的优化框架, 包含三个关键组件: (1) 元生成器 (MGAGENT, § 3.4); (2) 预测器训练器 (FTAGENT, § 3.5); (3) 框架提议器 (HPAGENT, § 3.6)。其中, HPAGENT 在智能体优化过程中优化 MGAGENT 的框架, 该框架在部署和推理阶段将被移除 (§ 3.7)。

3.3 框架优化问题

为使元铸造者 (METACASTER) 能够生成领域特定的训练数据, 我们构建了一个时间序列数据集语料库 $\mathcal{C}_{\text{har}} = \{\mathcal{D}^m, \mathcal{C}_m\}_{m=1}^M$ 用于驾驭 (Harness) 优化, 其中 $\mathcal{D}^m = \{(\mathbf{X}_i, \mathbf{Y}_i)\}_{i=1}^{N_m}$ 表示来自第 m 个领域的时间序列, $\mathcal{C}_m$ 为其关联的上下文。该语料库涵盖 M 个不同领域, 以提升元铸造者的跨领域泛化能力。如图 2 所示, 每个数据集 $\mathcal{D}^m$ 被划分为训练集、验证集和测试集 $\mathcal{D}_{\text{tr}}^m, \mathcal{D}_{\text{val}}^m$ 与 $\mathcal{D}_{\text{te}}^m$, 代表来自第 m -th 个领域的真实数据。为模拟实际的少样本场景, 从 $\mathcal{D}_{\text{tr}}^m$ 中选取 K 个样本用于构建 $\mathcal{D}_{\text{sup}}^m$ 。随后, 元铸造者利用 $\{\mathcal{D}_{\text{sup}}^m, \mathcal{C}_m\}$ 生成 $\{\bar{\mathcal{D}}_{\text{tr}}^m, \bar{\mathcal{D}}_{\text{val}}^m\}$ 。目标是缩小在

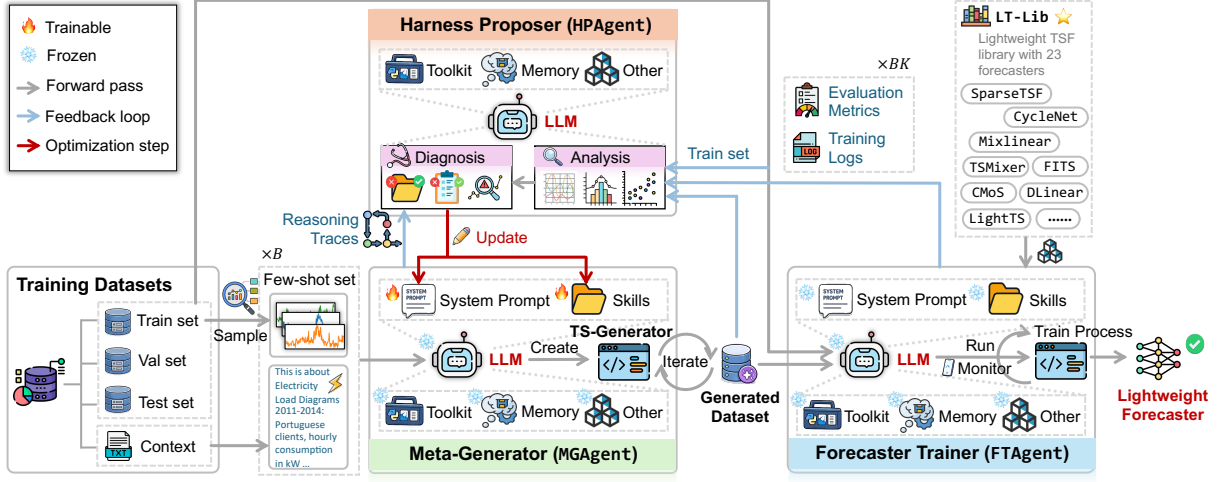


图 2: 所提出的元铸造者系统的驾驭优化框架示意图。

$\{\mathcal{D}_{tr}^m, \mathcal{D}_{val}^m\}$ 上训练的预测器与在 $\{\bar{\mathcal{D}}_{tr}^m, \bar{\mathcal{D}}_{val}^m\}$ 上训练的预测器在同一测试集 $\mathcal{D}_{te}^m$ 上评估的性能差距。形式上, 设 θ 为元铸造者中可训练的驾驭组件, f_l^m ($\bar{f}_l^m$) 为使用 $\{\mathcal{D}_{tr}^m, \mathcal{D}_{val}^m\}$ ($\{\bar{\mathcal{D}}_{tr}^m, \bar{\mathcal{D}}_{val}^m\}$) 训练的预测器, 则驾驭优化问题为

$$\min_{\theta} \mathbb{E}_{1 \leq m \leq M, 1 \leq l \leq L} (\delta(\omega(f_l^m), \omega(\bar{f}_l^m))) \quad (1)$$

其中 $\omega(\cdot)$ 是评估 $\mathcal{D}_{te}^m$ 上预测误差的度量, $\delta(\cdot, \cdot)$ 是两个度量之间差异的测度。

3.4 元生成器 (MGAGENT)

给定少样本集 $\{\mathcal{D}_{sup}^m, C_m\}$, MGAGENT, 其目标是生成 $\bar{\mathcal{D}}^m$, 即

$$\bar{\mathcal{D}}^m = \text{MGAGENT}(\mathcal{D}_{sup}^m, C_m) \quad (2)$$

该生成结果将被划分为 $\{\bar{\mathcal{D}}_{tr}^m, \bar{\mathcal{D}}_{val}^m\}$ by FTAGENT (见第 3.5 节)。此处, C_m 提供语义线索, 引导元生成器使用与第 m 个领域相关的方法生成符合领域规范信号。如图 2 所示, 元生成器以一个 (可替换的) 大语言模型 (LLM) 为核心。该大语言模型并非直接生成多元时间序列 (MTS), 而是利用其驾驭组件创建一个时间序列生成器 (TS-Generator) 程序, 该程序整合了与第 m 个领域相关的领域特定知识、规则和模型, 从而规避了大语言模型在直接时间序列推理方面的能力局限 (Merrill 等, 2024)。相反, 元生成器利用大语言模型在规划、推理和编码方面的优势, 来编排更适合时间序列推理的工具, 因此得名“元 (Meta-)”前缀。在接收到

$\{\mathcal{D}_{sup}^m, C_m\}$, MGAGENT 后, 元生成器 (1) 分析 $\mathcal{D}_{sup}^m$ 中的多元时间序列, (2) 创建时间序列生成器以生成 $\mathcal{D}^m$, 以及 (3) 执行质量检查。

若所有检查均通过, 则接受 $\mathcal{D}^m$ 。否则, 返回步骤 (2) 修改时间序列生成器。在此过程中, 驾驭组件向大语言模型暴露若干组件, 包括系统提示、技能、工具包、长期记忆以及其他执行中间件。系统提示定义了元生成器代理 (MGA-)

GENT 的行为、目标、约束和工具使用规则。技能 (Skills) (Li 等人, 2026b) 是可复用模块, 包含针对公式 (2) 中时间序列生成任务的指令、元数据和工具绑定。工具包为 Python 和网络搜索等工具提供应用程序编程接口 (API), 而长期记忆 (Long-term Memory) 存储已生成数据的信息, 以支持步骤 (2) 和 (3) 中的迭代审查与改进。在 Harness 中, 系统提示和技能定义了 MGAGENT 的关键行为。因此, 本文冻结大语言模型 (LLM), 并将其用作可训练参数 θ 。不同于 Harness 工程 (Li 等人, 2026a), θ 由 HPAGENT (§ 3.6) 在外层循环中编辑, 以自动优化公式 (1)。

3.5 预测器训练器 (FTAGENT)

FTAGENT 根据 $\{\mathcal{D}_{tr}^m, \mathcal{D}_{val}^m\}$ 的划分大小, 将 $\bar{\mathcal{D}}^m$ 分割为 $\{\bar{\mathcal{D}}_{tr}^m, \bar{\mathcal{D}}_{val}^m\}$, 协调计算资源以训练 $\mathcal{F}$ 中的预测器, 并在 $\mathcal{D}_{te}^m$ 上评估它们, 生成训练后的预测器、评估度量和训练报告:

$$\begin{aligned} & \{\{f_l^m, \bar{f}_l^m, \omega(f_l^m), \omega(\bar{f}_l^m)\}_{l=1}^L, R\} \\ &= \text{FTAGENT}(\{\mathcal{D}_{tr}^m, \mathcal{D}_{val}^m, \mathcal{D}_{te}^m\}, \{\bar{\mathcal{D}}_{tr}^m, \bar{\mathcal{D}}_{val}^m\}) \end{aligned} \quad (3)$$

其中 $\omega(\cdot)$ 是评估度量, 如均方误差 (MSE) 和平均绝对误差 (MAE), R 表示训练报告。轻量级时间序列预测库 (LT-LIB)。为构建 $\mathcal{F}$, 本文收集了 2022 年至 2026 年间提出的 23 个最先进的轻量级预测器, 包括

线性层模型（例如 MixLinear (Ma 等人, 2024)）、基于多层感知机 (MLP) 的预测器（例如 TSMixer (Chen 等人, 2023)）以及频域模型（例如 FITS (Xu 等人, 2024)）。如附录 C.1 所列，它们的规模远小于最先进的基于变换器 (Transformer) 的预测器（例如 PatchTST (Nie 等人, 2023): $\sim 3.2M$ ）和时间序列基础模型 (TSFM)（例如 Chronos (Ansari 等人, 2024): $\sim 700M$, We)。本文将它们编译到 LT-LIB 中，并采用统一接口以方便 FTAGENT 中的训练调用。如图 2 所示，FTAGENT 编写程序以网格搜索超参数的方式训练每个预测器。它将训练任务（即（预测器、超参数、数据集）三元组）组织成队列，并分配可用的图形处理器 (GPU) 以实现最大并行执行。在训练过程中，它监控进程，遇到错误时予以解决，并在无需人工干预的情况下恢复中断的任务。当队列清空时，此过程结束。FTAGENT 的 Harness 与 MGAGENT 的 Harness 共享相同的组件，并额外拥有对 LT-LIB 的访问权限。然而，与 MGAGENT 不同，其 Harness 未经优化，因为训练过程相对标准，且对最终预测器的影响小于生成的数据集 $\bar{D}^m$ 。

3.6 Harness 提议器 (HPAGENT)

HPAGENT 是元框架 (Lee 等, 2026)，用于配置 MGAGENT 的框架以优化式 (1)。在从 FTAGENT 接收到 $\{\omega(f_l^m), \omega(\bar{f}_l^m)\}_{l=1}^L$ 后，HPAGENT 使用基于铰链损失的测度评估式 (1)：

$$\delta(\omega(f_l^m), \omega(\bar{f}_l^m)) = \max \left\{ \frac{\omega(\bar{f}_l^m) - \omega(f_l^m)}{\omega(f_l^m)}, 0 \right\} \quad (4)$$

当在生成的 $\bar{D}_r$ 上训练的预测器的预测误差 $\omega(\bar{f}_l^m)$ 大于在真实数据集 D_r 上训练的预测器的 $\omega(f_l^m)$ 时，该测度会引入惩罚，表明 $\bar{D}_r$ 需要改进。否则，当 $\bar{f}_l^m$ 的表现优于 f_l^m 时，式 (4) 不会引入任何惩罚。如图 2 所示，HPAGENT 通过三个阶段监督整个流程：（1）自规划分析，收集关于 MGAGENT 和 FTAGENT 性能的证据，包括式 (1) 中的损失、 D_r 与 $\bar{D}_r$ 之间的统计差异（例如，MMD），以及来自日志 R 的训练问题；（2）诊断，从 MGAGENT 的推理迹、系统提示和技能中识别性能下降的潜在原因，然后确定如何更新框架 θ of MGAGENT；以及

（3）更新，编辑 θ 。实际上，HPAGENT 充当式 (1) 的优化器，由其系统提示定义。因此，HPAGENT 的生命周期是整个优化过程，包含多个更新 θ 的训练轮次，而 MGAGENT 和 FTAGENT 各自在单个训练轮次内运行。因此，HPAGENT 更依赖于长期记忆，该记忆跨训练轮次存储数据集快照、评估度量、训练日志、分析/诊断结果、 θ 更新日志等。这种记忆能够回滚有害更新，并允许 HPAGENT 在优化结束时输出最佳的 θ 。

3.7 训练与推理

框架训练。我们在附录 A 中总结了优化过程。在每个训练轮次中，我们抽取 $K \sim$ 均匀分布 $([10, 50])$ 。值得注意的是，我们没有将 K 样本支持集 $\{D_{sup}^m, C_m\}$ 单独输入 METACASTER，而是从语料库 C_{char} 中采样一批 B 个数据集以提取 B 个支持集，并让 METACASTER 同时处理它们。这种扩展使 HPAGENT 能够评估跨数据集的泛化能力，并减轻对任何特定数据集的过拟合。

推理。优化完成后，HPAGENT 输出最佳 θ^* 并被丢弃。MGAGENT（包括 θ^* ）和 FTAGENT（包括 LT-LIB）的 Harness 被保留，而其 LLM 被移除。对于新的目标域 g ，这些 Harness 被附加到用户选择的 LLM API 上，以实例化 MGAGENT 和 FTAGENT。给定 K 个样本 D_{sup}^g 和上下文 C_g ，智能体执行图 2 中的前向过程，生成训练好的预测器 $\bar{f}_*^g$ 。在 TSF 阶段，仅需维护 $\bar{f}_*^g$ ，而 MGAGENT 和 FTAGENT 被丢弃，如图 1(c) 所示。因此，整个下游部署和推理过程可以使用能够运行 Harness 和 LT-LIB 框架的低成本设备完成，比依赖 GPU 密集型 TSFM 更高效。

4 实验

4.1 实验设置

数据集。为评估 METACASTER，我们采用 GIFT-Eval 基准 (Aksu 等, 2024)，该基准此前已用于训练 TSFM (Woo 等, 2024)。我们使用涵盖 9 个领域（如能源、天气、交通）的 18 个数据集，划分为包含 8 个数据集的训练语料库（§ 3.3 中的 *i.e.*, C_{char} ）、一个域内 (IND)

	Dataset	Ours	Generation Models					Augmentation Methods				References	
		METACASTER	TimeDP	VerbalTS	T2S	DiffTS	TimeVAE	Repeat	Bootstrap	Jitter	MagWarp	$\mathcal{D}_{\text{sup}}^m$	$\mathcal{D}_{\text{tr}}^m$
IND ($K = 10$)	ETTm1	0.376	1.049	1.004	0.922	1.154	0.749	0.781	0.748	0.746	1.919	0.687	0.316
	Electricity	0.300	0.288	1.061	1.150	0.365	0.328	0.380	0.379	0.375	7.057	0.721	0.121
	Seattle	1.522	1.420	1.800	1.792	1.643	1.759	1.955	1.849	1.925	43.24	2.341	1.024
	SZTaxi	0.104	0.124	0.116	0.118	0.111	0.114	0.134	0.118	0.123	1.122	0.187	0.096
	Sales	2.616	2.587	2.419	2.758	4.323	2.795	4.175	2.949	2.946	4.321	3.004	2.927
	Bitbrains	0.068	0.079	0.098	0.094	0.121	0.109	0.197	0.149	0.159	0.826	0.152	0.150
	Solar	0.158	0.208	0.513	0.552	0.238	0.237	0.278	0.272	0.272	0.446	0.440	0.157
OOD	Saugeen	1.960	1.492	1.474	1.578	2.281	2.705	3.119	2.887	2.908	3.893	1.859	1.183
	USbirths	0.702	2.690	1.501	1.540	1.775	0.607	0.752	0.715	0.714	12.08	1.204	0.250
	M4 [†]	2.093	2.178	2.331	2.444	2.688	2.993	3.250	3.101	3.057	8.654	2.628	2.330
IND ($K = 30$)	ETTm1	0.345	1.049	1.027	0.910	1.211	0.684	0.714	0.664	0.662	0.943	0.602	0.316
	Electricity	0.226	0.287	1.055	1.140	0.363	0.268	0.305	0.287	0.290	1.644	0.657	0.121
	Seattle	1.177	1.414	1.806	1.777	1.673	2.158	1.738	1.656	1.622	21.15	1.785	1.024
	SZTaxi	0.114	0.121	0.116	0.115	0.112	0.156	0.162	0.141	0.135	1.626	0.150	0.096
	Sales	2.362	2.479	2.455	2.942	4.439	2.598	3.603	3.131	3.158	3.422	2.611	2.927
	Bitbrains	0.125	0.084	0.097	0.095	0.130	0.114	0.463	0.469	0.489	0.777	0.137	0.150
	Solar	0.152	0.220	0.515	0.576	0.240	0.234	0.254	0.252	0.248	0.356	0.379	0.157
OOD	Saugeen	1.464	1.495	1.480	1.593	2.286	2.019	2.419	2.266	2.277	2.620	1.547	1.183
	USbirths	0.533	1.980	1.509	1.536	1.732	0.474	0.546	0.524	0.525	5.107	0.803	0.250
	M4 [†]	2.112	2.248	2.394	2.522	2.663	3.402	3.126	3.109	3.169	10.321	2.950	2.330
IND ($K = 50$)	ETTm1	0.267	1.063	1.035	0.931	1.284	0.548	0.582	0.556	0.546	0.592	0.456	0.316
	Electricity	0.191	0.279	1.053	1.197	0.352	0.227	0.244	0.234	0.233	1.287	0.472	0.121
	Seattle	1.125	1.423	1.819	1.779	1.732	2.810	1.537	1.462	1.497	12.78	1.533	1.024
	SZTaxi	0.110	0.120	0.114	0.113	0.113	0.318	0.230	0.222	0.216	0.352	0.143	0.096
	Sales	2.835	2.605	2.474	2.852	4.459	2.683	2.778	2.590	2.560	3.039	2.501	2.927
	Bitbrains	0.119	0.097	0.099	0.095	0.124	0.155	0.929	0.976	0.987	1.214	0.119	0.150
	Solar	0.149	0.234	0.520	0.553	0.250	0.547	0.234	0.231	0.229	0.333	0.296	0.157
OOD	Saugeen	1.415	1.464	1.482	1.591	2.291	1.767	2.462	2.380	2.371	2.528	1.515	1.183
	USbirths	0.424	2.366	1.509	1.531	1.687	0.367	0.396	0.380	0.376	5.787	0.574	0.250
	M4 [†]	1.916	2.157	2.455	2.444	2.896	3.279	3.330	3.300	3.282	12.210	2.560	2.330
Wins (of 30)		19	3	3	1	1	3	0	0	0	0	—	—

表 1: 在 $K \in \{10, 30, 50\}$ 的 IND 和 OOD 语料库上, 以 MSE 为指标的 TSF 性能比较。MSE 越低越好。红色 (蓝色) 值表示每行最佳 (次佳) MSE。[†] M4 使用实例归一化 MSE 以应对较大的分布偏移。MAE 结果见附录 E。

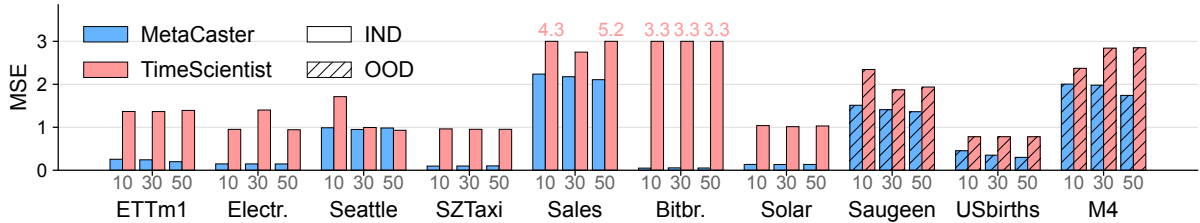


图 3: 比较智能体流水线 METACASTER 和 TimeScientist 所选 (训练后) 的预测器。

测试语料库 (7 个数据集) 和一个域外 (OOD) 测试语料库 (3 个数据集)。IND 语料库与 $\mathcal{C}_{\text{har}}$ 共享领域, 而 OOD 语料库涵盖未见领域以评估泛化能力。所有测试数据集与 $\mathcal{C}_{\text{har}}$ 不相交。每个数据集 $\mathcal{D}^m$ 具有文本上下文 $\mathcal{C}_m$ 。时间序列按时间顺序划分为 80%/10%/10% 的训练/验证/测试集。遵循标准协议 (Nie 等, 2023), 回看窗口 T 设为 336, 预测范围 H 设为 192。完整的数据集详情和避免数据泄漏的方法见附录 B.1。

基线方法。 本文将元铸造者 (METACASTER) 与最相关的最先进方法进行比较, 包括时间序列生成模型: (1) 时间变分自编码器 (TimeVAE) (Desai 等, 2021), (2) 扩散时间序列 (DiffTS) (Yuan 和 Qiao, 2024), (3) 文本到序列 (T2S) (Ge 等, 2025), (4) 时间扩散过程 (TimeDP) (Huang 等, 2025b),

(5) 语言时间序列 (VerbalTS) (Gu 等, 2025); 时间序列增强技术 (Iwana 和 Uchida, 2021; Luo 等, 2023); (6) 重复 (Repeat), (7) 自助法 (Bootstrap), (8) 抖动 (Jitter), (9) 幅度扭曲 (MagWarp); 预训练时间序列基础模型: (10) 克罗诺斯 (Chronos) (Ansari 等, 2024), (11) 莫伊莱 (Moirai) (Woo 等, 2024), (12) 视觉时间序列 (VisionTS) (Chen 等, 2024),

(13) 时间大语言模型 (Time-LLM) (Jin 等, 2024); 以及最先进的智能体流程: (14) 时间科学家 (TimeScientist) (Zhao 等, 2025)。

这些方法的详细信息见附录 B.2。

实验设置。 本文从每个测试数据集中抽取 $K \in \{10, 30, 50\}$ 个样本构成 $\mathcal{D}_{\text{sup}}^m$, 并与 $\mathcal{C}_m$ 一起输入到对比方法中。生成模型和增强方法使用 $\{\mathcal{D}_{\text{sup}}^m, \mathcal{C}_m\}$ 生成大小为 N'_m 的 $\bar{\mathcal{D}}^m$, 用于训练本文长期时间序列库 (LT-LIB) 中的预测器。时间序列基础模型使用 $\mathcal{D}_{\text{sup}}^m$ 进行微调。时间科学家 (TimeScientist) 使用 $\mathcal{D}_{\text{sup}}^m$ 直接训练预测器, 因为它无法生成时间序列。

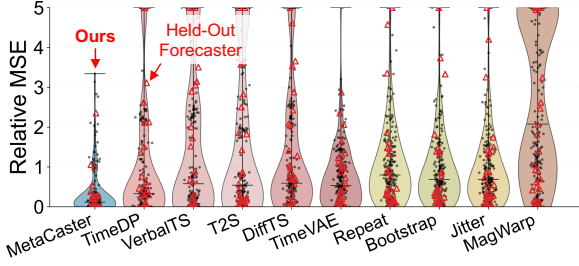


图 4: 性能分布。每个点代表一个预测器 $\bar{f}_l^m$ 在测试集 $\mathcal{D}_{te}^m$ 上的性能, 覆盖长期时间序列库 (LT-LIB) 中的所有预测器以及分布内 (IND) 和分布外 (OOD) 语料库中的所有数据集。

对于我们的元铸造者 (METACASTER), 我们设置批大小 $B = 8$, 并使用 GPT-5.4 (Lee 等人, 2026) 作为默认大语言模型 (LLM)。我们还在消融实验中测试了不同的大语言模型 (表 2)。对于所有能生成 $\bar{\mathcal{D}}^m$ 的方法, 大小 N_m' 设置为 $|\mathcal{D}_{tr}^m| + |\mathcal{D}_{val}^m|$ 。遵循 (Nie 等人, 2023; Tan 等人, 2024), 我们使用均方误差 (MSE) 和平均绝对误差 (MAE) 来评估时间序列预测 (TSF) 性能。

4.2 实验结果

表 1 报告了比较生成模型、增强方法和元铸造者的均方误差结果。我们从 LT-LIB 中随机保留 3 个预测器, 并评估剩余 20 个的平均性能; 保留的预测器用于评估元铸造者对未见预测器的泛化能力 (图 4)。平均绝对误差结果见附录 E, 附录 F 报告了比较方法训练的 20 个预测器中表现最佳的预测器。表 1 还报告了使用 K 次 $\mathcal{D}_{sup}^m$ 和完整集 $\mathcal{D}_{tr}^m$ 训练预测器的结果, 分别作为性能下限和上限参考。从表 1 中, 我们强调几点观察: (1) 元铸造者在大多数情况下优于生成/增强基线, 证明了为预测优化数据生成的好处; (2) 元铸造者的性能随着 K 的增大而提升, 显示出有效的少样本利用; (3) 当 $K \geq 30$, M 元铸造者接近甚至超过 $\mathcal{D}_{tr}^m$ 时, 表明原始数据可能含有噪声, 优化数据可以改善训练; (4) 在 $K = 10$, M 元铸造者保持竞争力的极端情况下; (5) 尽管难度增加, 元铸造者对分布外 (OOD) 数据集泛化良好, 通常优于基线。图 4 显示了表 1 中模型训练的单个预测器的均方误差分布, 按上限参考 ($\mathcal{D}_{tr}^m$) 归一化。

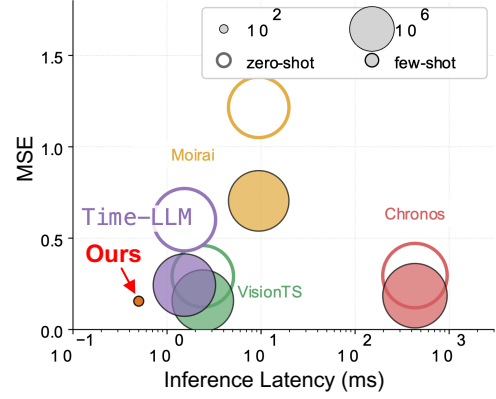


图 5: 在 Solar 数据集上比较元铸造者与时间序列基础模型 (TSFM)。完整结果见附录 E。

(即公式 (4))。元铸造者产生更多高质量且方差更低的预测器。此外, 它比基线更好地泛化到保留的预测器。图 3 比较了元铸造者在 $K \in 10, 30, 50$ 下与时间科学家 (TimeScientist) 的表现。没有数据生成能力, 时间科学家难以训练可泛化的预测器, 其性能不随 K 扩展, 表明纯训练流程不足。附录 E 报告了时间科学家使用 LT-LIB 时的结果, 得出一致的结论。

与时间序列基础模型比较 图 5 在 $K = 30$ 条件下, 将元铸造者 (Meta-Caster) 与时间序列基础模型 (TSFM) 在太阳能数据集上进行了性能与成本的对比。部署后, 元铸造者训练并选择一个轻量级预测器进行推理, 不再产生额外的智能体开销——此处运行时选择的是混合线性模型 (MixLinear, 243 个参数)。相比之下, 时间序列基础模型从微调到推理始终成本高昂。在性能相当的情况下, 元铸造者相比时间序列基础模型实现了高达 $10^3 \times$ 的延迟降低和 $10^5 \times$ 的参数减少。

消融实验。表 2 展示了使用 $K = 30$ 平均均方误差 (MSE) 的消融实验, 其中元铸造者作为原始模型。在 (a) 中, 我们通过将式 (1) 替换为最大均值差异 (MMD) 和瓦瑟斯坦距离来直接对齐生成集与真实集 $\mathcal{D}_{tr}^m$, 研究式 (1) 中面向预测的 $\bar{\mathcal{D}}_t$ 优化的效果。在 (b) 中, 我们移除上下文线索 C_m 以评估其贡献。在 (c) 中, 我们考察不同大语言模型 (LLM) 的影响。从表 2 (a) 可以看出, 最小化数据分布差异通常会降低性能, 因为它与预测目标并不直接对齐。表 2 (b) 表明上下文线索 C_m 至关重要, 因为它们引导元生成智能体 (MGAGENT) 选择领域相关知识来生成时间序列。

Dataset ($\rightarrow$) Method ($\downarrow$)	ETTM1	Electricity	IND Corpus Seattle	SZTaxi	Sales	Bitbrains	Solar	OOD Corpus Saugeen	USBirths	M4	Overall
METACASTER	0.345	0.226	<u>1.177</u>	<u>0.114</u>	<u>2.362</u>	0.125	0.152	1.464	<u>0.533</u>	2.112	0.267
(a) Loss $\rightarrow$ MMD	0.708	0.157	1.204	0.150	3.581	0.575	0.254	2.339	0.403	2.556	0.764
(a) Loss $\rightarrow$ Wasserstein	0.702	0.284	1.734	0.152	3.553	0.483	0.254	2.406	0.535	3.105	0.940
(b) Remove context C_m	<u>0.386</u>	0.293	1.425	<u>0.144</u>	2.595	0.158	0.281	2.004	0.534	2.312	0.521
(c) LLM $\rightarrow$ Gemini-3.1-Pro	0.430	0.226	<u>1.177</u>	<u>0.114</u>	<u>2.362</u>	0.120	0.151	1.396	<u>0.533</u>	<u>2.129</u>	<u>0.288</u>
(c) LLM $\rightarrow$ Claude-Opus-4.7	0.494	0.226	1.189	<u>0.114</u>	<u>2.362</u>	0.151	<u>0.150</u>	1.491	0.541	<u>2.129</u>	0.321
(c) LLM $\rightarrow$ Qwen3.5-122B-A10B	0.494	0.223	<u>1.177</u>	<u>0.114</u>	<u>2.362</u>	0.199	0.152	1.729	<u>0.533</u>	<u>2.129</u>	0.366
(c) LLM $\rightarrow$ GPT-5.3-Codex	0.557	<u>0.209</u>	1.115	0.087	2.229	0.098	0.148	<u>1.465</u>	1.488	2.269	0.677

表 2: 以均方误差 (MSE) 衡量的消融分析。“总体”评估跨数据集的归一化均方误差 (式 (4))。

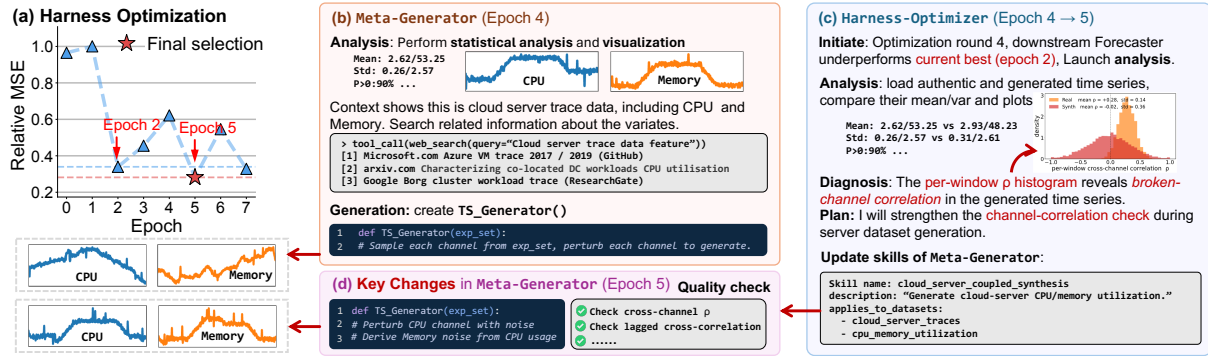


图 6: 元铸造者进行线束 (Harness) 优化的示意图: (a) 优化迭代过程中的铰链损失 (式 (4))。 (b) - (d) 在 alibaba_cluster_2018 数据集上从第 4 轮次到第 5 轮次元生成智能体和超参数 AGENT 的关键轨迹。

有趣的是, 表 2 (c) 显示不同大语言模型在许多情况下产生的结果相当, 这表明线束——而非大语言模型骨干——是元铸造者的关键因素, 这与先前的研究结果一致 (Li 等人, 2026a)。尽管 GPT-5.3-Codex 在某些情况下获胜, 但它在 ETTm1 和 USbirths 等数据集上表现不稳定, 导致总体结果较差; 因此, 我们采用 GPT-5.4 作为默认大语言模型, 因为它性能稳定。

进一步分析 我们评估了计算效率、令牌使用量、性能鲁棒性 (使用标准差)、性能变化 *w.r.t.* K 、排名靠前的预测器性能、生成时间序列的可视化以及数据分布, 这些内容见附录 C.3 和 F。

4.3 案例研究

图 6(a) 展示了在 8 个 Harness 优化轮次中铰链损失 (公式 (4)) 的演变过程, 其中 METACASTER 快速收敛, 并从第 5 轮次起选定最终 Harness。图 6(b)-(d) 展示了在 C_{har} MGA-AGENT 中 alibaba_cluster_2018 数据集上从第 4 轮次到第 5 轮次的 MGA-AGENT 和 HPAGENT 轨迹 (图 6(b)) 分析小样本示例 $\mathcal{D}_{\text{sup}}^m$, 检查统计信息, 检索关于 CPU 和内存两个变量的领域知识, 并构建 TS_Generator。HPAGENT (图 6(c)) 在第 4 轮次检测到性能下降, 通过

内存, 然后执行分析与诊断。分析使用与 MGA-推理轨迹中识别的问题相关的分析工具

GENT 和 FTAGENT 的训练日志。诊断识别出变量间相关性破坏为根本原因。随后更新 MGAGENT 的技能, 产生改进的 TS_Generator, 增强 CPU-内存相关性并增加一致性检查。因此, 第 5 轮次生成的时间序列比第 4 轮次更符合小样本示例。此示例展示了一个简单的 Harness 更新。实际上, METACASTER 的优化更为复杂, 它同时处理 $B = 8$ 个数据集集中的 C_{har} 。最终输出是训练好的轻量级预测器, 而非生成的时间序列。

5 结论

本文研究通过新颖的元 Harness 优化多智能体 METACASTER 实现轻量级预测器的小样本学习, 该框架自动化了时间序列生成、预测器训练、参数调优和模型选择的端到端流程。实验不仅验证了 METACASTER 的有效性, 还为深入探索智能体时间序列预测中的“智能体即工程师”范式奠定了基础。

局限性

本文研究在少样本场景下改进轻量级时间序列预测器这一具有挑战性但实用的问题，以实现快速部署，避免因昂贵的数据采集和隐私约束造成的延迟。这通过利用人工智能代理系统的知识和推理能力得以实现。然而，当前工作未处理极端零样本场景，即没有任何参考示例可用的情况。缺乏参考时间序列时，代理缺少目标域的基本统计基础，导致数据生成不可靠。这是基于时间序列生成的模型（Jeon 等，2022；Yuan 和 Qiao，2024；Naiman 等，2024；Huang 等，2025b；Gu 等，2025；Ge 等，2025）的常见局限性，并凸显了时间序列基础模型（TSFM）的优势，它们能在零样本场景下迁移预训练知识。相比之下，仅利用少量示例——在实践中通常是可行的——METACASTER 就能达到与 TSFM 相当的竞争力性能，同时保持显著更低的计算成本。此外，我们的实验覆盖了 18 个时间序列数据集，涉及有限的领域范围。我们计划将其扩展到现代 TSFM 的完整预训练语料库，以进一步改进 Harness 优化。同时，LT-LIB 目前包含我们迄今收集的一组最先进的轻量级预测器。随着新的轻量级预测器不断涌现，它可能并不完备。在未来工作中，我们将持续更新 LT-LIB 以增强 METACASTER 系统。

致谢

这项工作部分得到了 NEC Laboratories America 的研究馈赠和 NVIDIA 学术资助计划的支持。

参考文献

- Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. 2024. Gift-eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*.
- Alibaba Group. 2018. Alibaba cluster trace v2018. <https://github.com/alibaba/clusterdata>.
- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. 2024. Chronos: Learning the language of time series. *TMLR*.
- Alberto Ardid, David Dempsey, Corentin Caudron, Shane Cronin, Ben Kennedy, Társilo Girona, Diana Roman, Craig Miller, Sally Potter, Oliver D Lamb, and 1 others. 2025. Ergodic seismic precursors and transfer learning for short term eruption forecasting at data scarce volcanoes. *Nat. Commun.*
- Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small language models are the future of agentic ai. *arXiv preprint arXiv:2506.02153*.
- Verónica Bolón-Canedo, Laura Morán-Fernández, Brais Cancela, and Amparo Alonso-Betanzos. 2024. A review of green artificial intelligence: Towards a more sustainable future. *Neurocomputing*.
- Centers for Disease Control and Prevention. Flu-View: Outpatient illness surveillance (ILINet). <https://gis.cdc.gov/grasp/fluview/fluportaldashboard.html>.
- Centers for Disease Control and Prevention, NCHS. National vital statistics system: Births data. <https://www.cdc.gov/nchs/nvss/births.htm>.
- Mouxian Chen, Lefei Shen, Zhuo Li, Xiaoyun Wang, Jianling Sun, and Chenghao Liu. 2024. Visions: Visual masked autoencoders are free-lunch zero-shot time series forecasters. *arXiv preprint arXiv:2408.17253*.
- Si-An Chen, Chun-Liang Li, Nate Yoder, Sercan O. Arik, and Tomas Pfister. 2023. Tsmixer: An all-mlp architecture for time series forecasting. *TMLR*.
- Sarkar Snigdha Sarathi Das, Palash Goyal, Mihir Parmar, Nanyun Peng, Vishy Tirumalashetty, Chun-Liang Li, Rui Zhang, Jinsung Yoon, and Tomas Pfister. 2026. Nexus: An agentic framework for time series forecasting. *arXiv preprint arXiv:2605.14389*.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. 2021. Timevae: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*.
- Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. TSMixer: Lightweight MLP-mixer model for multivariate time series forecasting. In *KDD*.
- Jingru Fei, Kun Yi, Wei Fan, Qi Zhang, and Zhen-dong Niu. 2025. Amplifier: Bringing attention to neglected low-energy components in time series forecasting. In *AAAI*.

- Azul Garza and Renée Rosillo Garcia. 2025. Timecopilot. In *NeurIPS Workshop on Recent Advances in Time Series Foundation Models*.
- Yunfeng Ge, Jiawei Li, Yiji Zhao, Haomin Wen, Zhao Li, Meikang Qiu, Hongyan Li, Ming Jin, and Shirui Pan. 2025. T2s: High-resolution time series generation with text-to-series diffusion models. In *IJCAI*.
- Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I Webb, Rob J Hyndman, and Pablo Montero-Manso. 2021. Monash time series forecasting archive. In *NeurIPS Datasets and Benchmarks*.
- Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. 2023. Large language models are zero-shot time series forecasters. In *NeurIPS*.
- Shuqi Gu, Chuyue Li, Baoyu Jing, and Kan Ren. 2025. Verbalts: Generating time series from texts. In *ICML*.
- Tong Guan, Zijie Meng, Dianqi Li, Shiyu Wang, Chao-Han Huck Yang, Qingsong Wen, Zuozhu Liu, Sabato Marco Siniscalchi, Ming Jin, and Shirui Pan. 2026a. Timeomni-1: Incentivizing complex reasoning with time series in large language models. In *ICLR*.
- Tong Guan, Sheng Pan, Johan Barthelemy, Zhao Li, Yujun Cai, Cesare Alippi, Ming Jin, and Shirui Pan. 2026b. Timeomni-v1: Unified models for time series understanding and generation. *arXiv preprint arXiv:2602.17149*.
- Keith W Hipel and A Ian McLeod. 1994. *Time Series Modelling of Water Resources and Environmental Systems*. Elsevier.
- Qihe Huang, Zhengyang Zhou, Kuo Yang, Zhongchao Yi, Xu Wang, and Yang Wang. 2025a. TimeBase: The power of minimalism in efficient long-term time series forecasting. In *ICML*.
- Yu-Hao Huang, Chang Xu, Yueying Wu, Wu-Jun Li, and Jiang Bian. 2025b. Timedp: Learning to generate multi-domain time series with domain prompts. In *AAAI*.
- Brian Kenji Iwana and Seiichi Uchida. 2021. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*.
- Gunjan Jalori, Preetika Verma, and Sercan Ö. Arık. 2025. Flairr-ts: Forecasting llm-agents with iterative refinement and retrieval for time series. In *Findings of EMNLP*.
- Jinsung Jeon, Jeonghak Kim, Haryong Song, Seunghyeon Cho, and Noseong Park. 2022. Gt-gan: General purpose time series synthesis with generative adversarial networks. In *NeurIPS*.
- Yushan Jiang, Kanghui Ning, Zijie Pan, Xuyang Shen, Jingchao Ni, Wenchao Yu, Anderson Schneider, Haifeng Chen, Yuriy Nevmyvaka, and Dongjin Song. 2025. Multi-modal time series analysis: A tutorial and survey. In *KDD*.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. 2024. Time-llm: Time series forecasting by reprogramming large language models. In *ICLR*.
- Yaxuan Kong, Yiyuan Yang, Yoontae Hwang, Wenjie Du, Stefan Zohren, Zhangyang Wang, Ming Jin, and Qingsong Wen. 2025. Time-mqa: Time series multi-task question answering with context enhancement. In *ACL*.
- Irena Koprinska, Dengsong Wu, and Zheng Wang. 2018. Convolutional neural networks for energy time series forecasting. In *IJCNN*.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.
- Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Jianyang Gu, Yingqiang Ge, Weijie Xu, Xi Fang, Xiang Xu, Tianchen Zhao, Youngeun Kim, Tianyang Wang, Jihun Hamm, Smita Krishnaswamy, Jun Huan, and Chandan Reddy. 2026a. Agent harness engineering: A survey. <https://openreview.net/pdf?id=eONq7FdiHa>.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, and 1 others. 2026b. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.
- Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *ICLR*.
- Zhe Li, Shiyi Qi, Yiduo Li, and Zenglin Xu. 2023. Revisiting long-term time series forecasting: An investigation on linear mapping. *arXiv preprint arXiv:2305.10721*.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, and Tao Gui. 2026. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*.
- Shengsheng Lin, Weiwei Lin, Xinyi Hu, Wentai Wu, Ruichao Mo, and Haocheng Zhong. 2024a. Cyclenet: Enhancing time series forecasting through modeling periodic patterns. In *NeurIPS*.
- Shengsheng Lin, Weiwei Lin, Wentai Wu, Haojun Chen, and Junjie Yang. 2024b. Sparsetsf: Modeling long-term time series forecasting with 1k parameters. In *ICML*.
- Peiyuan Liu, Hang Guo, Tao Dai, Naiqi Li, Jigang Bao, Xudong Ren, Yong Jiang, and Shu-Tao Xia. 2025a. Calf: Aligning llms for time series forecasting via cross-modal fine-tuning. In *AAAI*.

- Peiyuan Liu, Beiliang Wu, Yifan Hu, Naiqi Li, Tao Dai, Jigang Bao, and Shu-Tao Xia. 2025b. TimeBridge: Non-stationarity matters for long-term time series forecasting. In *ICML*.
- Yong Liu, Guo Qin, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. 2024. Autotimes: Autoregressive time series forecasters via large language models. In *NeurIPS*.
- Yong Liu, Guo Qin, Zhiyuan Shi, Zhi Chen, Caiyin Yang, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. 2025c. Sundial: A family of highly capable time series foundation models. In *ICML*.
- Dongsheng Luo, Wei Cheng, Yingheng Wang, Dongkuan Xu, Jingchao Ni, Wenchao Yu, Xuchao Zhang, Yanchi Liu, Yuncong Chen, Haifeng Chen, and 1 others. 2023. Time series contrastive learning with information-aware augmentations. In *AAAI*.
- Aitian Ma, Dongsheng Luo, and Mo Sha. 2024. MixLinear: Extreme low resource multivariate time series forecasting with 0.1K parameters. *arXiv preprint arXiv:2410.02081*.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2020. The M4 competition: 100,000 time series and 61 forecasting methods. *Int. J. Forecast.*
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2022. The M5 competition: Background, organization, and implementation. *Int. J. Forecast.*
- Mike A Merrill, Mingtian Tan, Vinayak Gupta, Thomas Hartvigsen, and Tim Althoff. 2024. Language models still struggle to zero-shot reason about time series. In *Findings of EMNLP*.
- Mohammad Amin Morid, Olivia R Liu Sheng, and Joseph Dunbar. 2023. Time series prediction using deep learning methods in healthcare. *ACM TMIS*.
- Ilan Naiman, Nimrod Berman, Itai Pemper, Idan Arbiv, Gal Fadlon, and Omri Azencot. 2024. Utilizing image transforms and diffusion models for generative modeling of short and long time series. In *NeurIPS*.
- National Renewable Energy Laboratory. 2006. Solar power data for integration studies. <https://www.nrel.gov/grid/solar-power-data.html>.
- Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A time series is worth 64 words: Long-term forecasting with transformers. In *ICLR*.
- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, and 1 others. 2026. Code as agent harness. *arXiv preprint arXiv:2605.18747*.
- Zijie Pan, Yushan Jiang, Sahil Garg, Anderson Schneider, Yuriy Nevmyvaka, and Dongjin Song. 2024. S2ip-llm: Semantic space informed prompt learning with llm for time series forecasting. In *ICML*.
- ChengAo Shen, Wenchao Yu, Ziming Zhao, Dongjin Song, Wei Cheng, Haifeng Chen, and Jingchao Ni. 2025a. Multi-modal view enhanced large vision models for long-term time series forecasting. In *NeurIPS*.
- ChengAo Shen, Ziming Zhao, Hanghang Tong, Dongjin Song, Dongsheng Luo, Qingsong Wen, and Jingchao Ni. 2025b. Svtime: Small time series forecasting models informed by "physics" of large vision model forecasters. *arXiv preprint arXiv:2510.09780*.
- Siqi Shen, Vincent van Beek, and Alexandru Iosup. 2015. Statistical characterization of business-critical workloads hosted in cloud datacenters. In *CCGrid*.
- Haotian Si, Changhua Pei, Jianhui Li, Dan Pei, and Gaogang Xie. 2025. CMoS: Rethinking time series prediction through the lens of chunk-wise spatial correlations. In *ICML*.
- Artyom Stitsyuk and Jaesik Choi. 2025. xPatch: Dual-stream time series forecasting with exponential seasonal-trend decomposition. In *AAAI*.
- Mingtian Tan, Mike A Merrill, Vinayak Gupta, Tim Althoff, and Thomas Hartvigsen. 2024. Are language models actually useful for time series forecasting? *NeurIPS*.
- Peiwan Tang and Weitai Zhang. 2025. Unlocking the power of patch: Patch-based MLP for long-term time series forecasting. In *AAAI*.
- Artur Trindade. 2015. Electricity-LoadDiagrams20112014. <https://archive.ics.uci.edu/dataset/321/electricityloadaddiagrams20112014>.
- University of Washington STAR Lab. Seattle inductive loop detector dataset. <https://github.com/zhilyongc/Seattle-Loop-Data>.
- Chengsen Wang, Qi Qi, Jingyu Wang, Haifeng Sun, Zirui Zhuang, Jinming Wu, Lei Zhang, and Jianxin Liao. 2025. Chattime: A unified multimodal time series foundation model bridging numerical and textual data. In *AAAI*.
- Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y. Zhang, and Jun Zhou. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. In *ICLR*.
- Qingsong Wen, Liang Sun, Fan Yang, Xiaomin Song, Jingkun Gao, Xue Wang, and Huan Xu. 2021. Time series data augmentation for deep learning: A survey. In *IJCAI*.
- Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified training of universal time series forecasting transformers. In *ICML*.
- Xingjian Wu, Junkai Lu, Zhengyu Li, Xiangfei Qiu, Jilin Hu, Chenjuan Guo, Christian S. Jensen, and Bin Yang. 2026. Timeart: Towards agentic time series reasoning via tool-augmentation. *arXiv preprint arXiv:2601.13653*.

- Mingyuan Xia, Chunxu Zhang, Zijian Zhang, Hao Miao, Qidong Liu, Yuanshao Zhu, and Bo Yang. 2025. TimeEmb: A lightweight static-dynamic disentanglement framework for time series forecasting. *arXiv preprint arXiv:2510.00461*.
- Zhijian Xu, Ailing Zeng, and Qiang Xu. 2024. Fits: Modeling time series with 10k parameters. In *ICLR*.
- Hao Xue and Flora D Salim. 2023. Promptcast: A new prompt-based learning paradigm for time series forecasting. *IEEE TKDE*.
- Kun Yi, Jingru Fei, Qi Zhang, Hui He, Shufeng Hao, Defu Lian, and Wei Fan. 2024. FilterNet: Harnessing frequency filters for time series forecasting. In *NeurIPS*.
- Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. 2019. Time-series generative adversarial networks. In *NeurIPS*.
- Fangxu Yu, Hongyu Zhao, and Tianyi Zhou. 2025. Ts-reasoner: Aligning time series foundation models with llm reasoning. *arXiv preprint arXiv:2510.03519*.
- Xinyu Yuan and Yan Qiao. 2024. Diffusion-ts: Interpretable diffusion for general time series generation. In *ICLR*.
- Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. 2022. Ts2vec: Towards universal representation of time series. In *AAAI*.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative ai by back-propagating language model feedback. *Nature*.
- Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting? In *AAAI*.
- Boya Zhang, Shuaijie Yin, Huiwen Zhu, and Xing He. 2026. FreqCycle: A multi-scale time-frequency analysis method for time series forecasting. *arXiv preprint arXiv:2603.09661*.
- Tianping Zhang, Yizhuo Zhang, Wei Cao, Jiang Bian, Xiaohan Yi, Shun Zheng, and Jian Li. 2022. Less is more: Fast multivariate time series forecasting with light sampling-oriented mlp structures. *arXiv preprint arXiv:2207.01186*.
- Xiaohan Zhang, Tian Gao, Mingyue Cheng, Bokai Pan, Ze Guo, Yaguo Liu, Xiaoyu Tao, and Qi Liu. 2025. Alphacast: A human wisdom-llm intelligence co-reasoning framework for interactive time series forecasting. *arXiv preprint arXiv:2511.08947*.
- Haokun Zhao, Xiang Zhang, Jiaqi Wei, Yiwei Xu, Yuting He, Siqi Sun, and Chenyu You. 2025. Timeseries-scientist: A general-purpose ai agent for time series analysis. *arXiv preprint arXiv:2510.01538*.
- Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, Min Deng, and Haifeng Li. 2020. T-GCN: A temporal graph convolutional network for traffic prediction. *IEEE TITS*.
- Siru Zhong, Weilin Ruan, Ming Jin, Huan Li, Qingsong Wen, and Yuxuan Liang. 2025. Time-vlm: Exploring multimodal vision-language models for augmented time series forecasting. In *ICML*.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI*.
- Pengfei Zhou, Yunlong Liu, Junli Liang, Qi Song, and Xiangyang Li. 2025. CrossLinear: Plug-and-play cross-correlation embedding for time series forecasting with exogenous variables. In *KDD*.
- Tian Zhou, Peisong Niu, Xue Wang, Liang Sun, and Rong Jin. 2023. One fits all: Power general time series analysis by pretrained lm. In *NeurIPS*.

A 算法

METACASTER 的元驾驭优化算法总结在算法 1 中。符号与 § 3 一致。

Algorithm 1: Meta-harness optimization of METACASTER

Input: (1) Corprurs $\mathcal{C}_{\text{har}} = \{\mathcal{D}^m, \mathcal{C}_m\}_{m=1}^M$; (2) LT-LIB $\mathcal{F} = \{f_1, \dots, f_L\}$; (3) number of epochs E ;
(4) batch size B

Output: Optimized harness θ^*

```

1  $\ell^* \leftarrow +\infty$  // Initialize loss function value
2  $\theta \leftarrow \theta^0$  // Initialize harness
3 for  $i = 1, \dots, E$  do
    /* Optimization loop */
4      $K \sim \text{Uniform}([10, 50])$ 
5     Draw a batch  $\mathcal{C}_b = \{\mathcal{D}^m, \mathcal{C}_m\}_{m=1}^B$ 
    /* Forward passes across datasets in a batch */
6     for  $\{\mathcal{D}^m, \mathcal{C}_m\} \in \mathcal{C}_b$  do
7          $\{\mathcal{D}_{\text{tr}}^m, \mathcal{D}_{\text{val}}^m, \mathcal{D}_{\text{te}}^m\} \leftarrow \text{Split}(\mathcal{D}^m)$  // train/validation/test split
8          $\mathcal{D}_{\text{sup}}^m \leftarrow K$  samples from  $\mathcal{D}_{\text{tr}}^m$  //  $K$ -shot support set
        /* Run MGagent (Eq. (2)) */
9          $\bar{\mathcal{D}}^m \leftarrow \text{MGAGENT}_{\theta}(\mathcal{D}_{\text{sup}}^m, \mathcal{C}_m)$ 
10         $\{\bar{\mathcal{D}}_{\text{tr}}^m, \bar{\mathcal{D}}_{\text{val}}^m\} \leftarrow \text{Split}(\bar{\mathcal{D}}^m)$ 
        /* Run FTagent (Eq. (3)) */
11         $\{\{f_l^m, \bar{f}_l^m, \omega(f_l^m), \omega(\bar{f}_l^m)\}_{l=1}^L, \mathbf{R}\} \leftarrow \text{FTAGENT}(\{\mathcal{D}_{\text{tr}}^m, \mathcal{D}_{\text{val}}^m, \mathcal{D}_{\text{te}}^m\}, \{\bar{\mathcal{D}}_{\text{tr}}^m, \bar{\mathcal{D}}_{\text{val}}^m\})$ 
12    end
13     $\ell \leftarrow \frac{1}{BL} \sum_{m=1}^B \sum_{l=1}^L \delta(\omega(f_l^m), \omega(\bar{f}_l^m))$  // The optimization loss Eq. (1)
    /* Run HPagent */
14     $\theta \leftarrow \text{HPAGENT}(\theta, \ell, \ell^*, \mathbf{R})$ 
15    if  $\ell < \ell^*$  then
16         $\ell^* \leftarrow \ell$ 
17         $\theta^* \leftarrow \theta$ 
18    end
19 end
```

B 数据集与基线

B.1 数据集

预训练式训练/测试划分。训练与评估数据集均取自 GIFT-Eval (Aksu 等人, 2024 年), 该数据集依据 Apache 2.0 许可证发布, 允许研究用途。为防止数据集泄漏, 本文遵循基础模型预训练所采用的数据隔离范式, 从两个互不相交的 GIFT-Eval 集合中分别抽取数据, 确保任何源序列均不会同时出现在两个阶段中。

解决数据泄漏问题。三层连续防护机制抵御测试源污染。首先, 训练与评估数据集分别取自两个互不相交的 GIFT-Eval 集合, 训练使用 Salesforce/GiftEvalPretrain, 评估使用 Salesforce/GIFT_Eval, 因此任何源序列都不会同时出现在两个阶段。其次, 传递给 MGAGENT 的上下文 $\mathcal{C}_m$ 在到达智能体之前已去除数据集标识符、源 URL 和基准名称, 仅保留领域标签和简短语义描述, 因此智能体无法利用数据集名称作为钩子来检索或模仿

来自任何预训练语料库的真实测试数据，它可能已经记忆。第三，我们手动审计了 MGAGENT 在留出运行中的运行时迹，并确认没有发出外部网络请求，也没有接触真实测试数据；综合仅由少样本支持集 $\mathcal{D}_{\text{sup}}^m$ 和去标识化上下文驱动。

训练数据采集。我们采用来自 Salesforce/GiftEvalPretrain (Aksu 等, 2024) 的 8 个数据集，涵盖 6 个领域类别（能源、交通、天气、健康、零售、云）；每个数据集的频率、通道数、长度、领域标签和来源列于表 3。其中两个 (cdc_fluview_ilinet, alibaba_cluster_trace_2018) 是多变量的（ $D = 5$ 和 $D = 2$ ）；其余六个从平行单变量序列总体中采样（ $D = 1$ ）。

Dataset	Freq.	D	N	T	Domain	Source
australian_electricity_demand	30min	1	5	231k	energy	(Godaheva et al., 2021)
solar_power	4s	1	1	7.4M	energy	(Godaheva et al., 2021)
PEMS_BAY	5min	1	325	52k	traffic	(Li et al., 2018)
traffic_hourly	1h	1	862	17.5k	traffic	(Godaheva et al., 2021)
weather	1d	1	3,010	var	weather	(Godaheva et al., 2021)
cdc_fluview_ilinet	1w	5	5	1.2k	health	(Centers for Disease Control and Prevention)
m5	1d	1	30,490	1.9k	retail	(Makridakis et al., 2022)
alibaba_cluster_trace_2018	1min	2	58k	2k+	cloud	(Alibaba Group, 2018)

表 3: 训练数据集概览。

测试集。本文从 Salesforce/GIFT_Eval (Aksu 等, 2024) 中选取 10 个数据集，划分为 7 个分布内（IND，大领域标签出现在训练集中）和 3 个分布外（OOD，完全未出现在训练集中）；每个数据集的频率、通道数、长度、领域标签和来源列于表 4。在 OOD 集合中，M4 是一个混合领域的日常子集；本文将其纳入以探究当领域信息模糊或缺失时模型是否仍然有效。

	Dataset	Freq.	D	N	T	Domain	Source
IND	ETTm1	15min	7	1	50k	energy	(Zhou et al., 2021)
	Electricity	1h	1	370	35.1k	energy	(Trindade, 2015)
	Seattle	1h	1	323	8.8k	traffic	(University of Washington STAR Lab)
	SZTaxi	15min	1	156	3.0k	traffic	(Zhao et al., 2020)
	Sales	1d	1	118	1.8k	retail	(Godaheva et al., 2021)
	Bitbrains	5min	2	1,250	8.6k	cloud	(Shen et al., 2015)
	Solar	1h	1	137	8.8k	energy	(National Renewable Energy Laboratory, 2006)
OOD	Saugeen	1d	1	1	23.7k	hydrology	(Hipel and McLeod, 1994)
	USbirths	1d	1	1	7.3k	demographics	(Centers for Disease Control and Prevention, NCHS)
	M4	1d	1	4,227	9.9k	mixed	(Makridakis et al., 2020)

表 4: 测试数据集概览。

难度谱系的覆盖范围。这 18 个数据集共同张成了 GIFT-Eval 上少样本预测难度的各个维度。采样频率从 solar_power 上的 4 秒光伏遥测到 cdc_fluview_ilinet, 上的每周健康指标，跨越六个数量级。变量数量 D 取值于 $\{1, 2, 5, 7\}$ ，涵盖了单变量和小型多通道场景。每个序列的长度从最短健康序列的 1.2 k 步到最密集遥测流的 7.4 M 步，跨度达五个数量级，使综合流程在单一基准内面临高度异构的场景。

每个数据集上的预测协议。在每个数据集上，原始序列按时间顺序以 80/10/10 的比例划分为训练集、验证集和测试集，使用训练集上估计的统计量对每个通道进行 z 分数归一化，并切分为 $(\mathbf{X}, \mathbf{Y})$ 对，回看窗口为 $T = 336$ 步，预测范围为 $H = 192$ 步；本文评估的所有方法均遵循这一约定。输入给 META-CASTER 及所有外部基线的少样本支持集 $\mathcal{D}_{\text{sup}}^m$ ，是从数据集 m 的训练集中抽取 K 个窗口得到的，主要结果（表 1）中 $K \in \{10, 30, 50\}$ ，且 $K \in \{10, 20, 30, 50, 100\}$ 。

在少样本扩展研究（附录 F）中。每个数据集与一个文本上下文 C_m 配对，该上下文概括其领域和语义线索，并与支持集捆绑，传递给任何使用文本条件的方法。

B.2 基线

本小节汇总了主表（表 1）中报告的所有基线，分为深度生成合成器和经典增强程序，以及两个参考水平，它们界定了每一行的动态范围。参考水平是 $\mathcal{D}_{\text{sup}}^m$ ，它直接在少样本支持集上训练每个预测器，不进行任何增强，作为无合成下限；以及 $\mathcal{D}_{\text{tr}}^m$ ，它在完整的历史训练分割上训练，作为数据丰富的上限，任何少样本方法都力求达到。为了完整性，我们另外描述了在 § 4.2 中比较的基础预测器以及最接近的智能体驱动参考 TimeScientist（图 3），这两者都不属于主表的少样本合成单元，但都出现在实验研究的其余部分。

深度生成基线。

- TimeDP（Huang 等人，2025b）：一种扩散模型，以从学习到的时间序列原型向量组装而成的领域提示为条件，其权重从目标领域的少样本中推断得出。
- VerbalTS（Gu 等人，2025）：一种扩散模型，通过桥接文本标记和时间潜变量的多焦点对齐模块，将非结构化文本描述映射到时间序列。
- T2S（Ge 等人，2025）：一种文本条件的扩散 Transformer，在长度自适应的 VAE 潜空间上使用流匹配进行训练，实现可变长度的文本到序列合成。
- Diffusion-TS（Yuan 和 Qiao，2024）：一种去噪扩散模型，采用编码器-解码器 Transformer，将样本分解为趋势和季节成分，并在傅里叶域损失下预测干净信号。
- TimeVAE（Desai 等人，2021）：一种变分自编码器，其解码器从高斯潜先验中采样的可解释水平、趋势和季节性模块组合输出。

经典增强基线。

- 重复：确定性地循环遍历少样本池，直至达到目标训练集规模。
- 自助法：有放回地重采样窗口，并添加按通道缩放的小幅零均值高斯噪声。
- 抖动（Iwana 和 Uchida，2021）：对每个重采样窗口的每个时间步和通道添加独立的零均值高斯噪声。
- 幅度扭曲（Iwana 和 Uchida，2021；Wen 等，2021）：将每个通道乘以一条平滑缩放曲线，该曲线通过对随机节点进行三次样条插值获得。

基础预测器。

- VisionTS（Chen 等，2024）：将预测重新表述为图像修复，通过将回看窗口排列成二维网格，并使用冻结的 ImageNet 预训练 MAE 解码被掩蔽的预测区间。
- Chronos（Ansari 等，2024）：将缩放后的时间序列值量化为固定词汇表，并在标记化序列上训练 T5 编码器-解码器，用于自回归概率预测。
- 莫伊莱（Moirai）（Woo 等人，2024）：一种掩码编码器变换器（Transformer），具有任意变量注意力和多补丁投影，在 LOTSA 语料库上预训练，可处理任意频率和通道数。
- 时间大语言模型（Time-LLM）（Jin 等人，2024）：冻结预训练的 GPT-2 主干网络，并将补丁嵌入重新编程为文本原型令牌，前置数据集描述作为提示前缀。

时间科学家 (TimeScientist)。时间科学家 (**TimeScientist**) (Zhao 等人, 2025) 是一种多智能体流水线, 由本地 Qwen2.5 主干驱动, 对每个序列进行诊断, 并按查询集成经典预测器 (ARIMA、ETS、随机森林)。我们将其与上述基础预测器区分开来, 因为它不提供单一的预训练权重矩阵; 相反, 大语言模型智能体在每个测试窗口被调用以组合和调整经典预测器, 使其成为与 METACASTER 的智能体驱动范式最接近的已发表参考。

C 实现细节

C.1 预测器池

四个家族中的二十三个轻量级预测器:

Family	Forecaster	Params	MACs (M)	Latency (ms)	Peak VRAM (MB)	Ref.
Linear	Vanilla Linear	65K	0.45	0.03	8.4	(Zeng et al., 2023)
	DLinear	129K	0.90	0.10	8.7	(Zeng et al., 2023)
	NLinear	65K	0.45	0.04	8.4	(Zeng et al., 2023)
	RLinear	65K	0.45	0.17	8.4	(Li et al., 2023)
	CrossLinear	2.5M	28.45	0.72	23.1	(Zhou et al., 2025)
	MixLinear	243	0.08	0.43	8.3	(Ma et al., 2024)
MLP	TSMixer	153K	1.66	0.42	10.1	(Chen et al., 2023)
	LightTS	74K	0.70	0.75	10.1	(Zhang et al., 2022)
	PatchMLP	2.5M	18.12	1.21	20.1	(Tang and Zhang, 2025)
	xPatch	770K	8.16	1.44	13.6	(Stitsyuk and Choi, 2025)
	CMoS	13K	0.36	1.09	9.3	(Si et al., 2025)
	PatchTSMixer	553K	20.83	1.12	14.4	(Ekambaram et al., 2023)
Freq/Filter	FITS	925	0.01	0.32	8.3	(Xu et al., 2024)
	CycleNet	65K	0.45	0.32	8.5	(Lin et al., 2024a)
	PaiFilter	136K	0.95	0.35	9.8	(Yi et al., 2024)
	TexFilter	179K	0.94	1.62	9.9	(Yi et al., 2024)
	FreqCycle	64K	0.43	0.78	9.5	(Zhang et al., 2026)
Mixing	TimeMixer	377K	50.08	0.80	65.0	(Wang et al., 2024)
	TimeBase	146	0.02	0.29	9.2	(Huang et al., 2025a)
	TimeBridge	36K	3.01	2.09	10.4	(Liu et al., 2025b)
	TimeEmb	308K	1.89	0.47	10.5	(Xia et al., 2025)
	Amplifier	327K	1.06	0.73	10.6	(Fei et al., 2025)
	SparseTSF	137	0.08	0.20	8.2	(Lin et al., 2024b)

表 5: LT-LIB 中的 23 个预测器, 按家族分组。

按架构逐一描述。为完整性起见, 本文按家族列出 LT-LIB 中每个预测器的核心架构思想; 文献引用与表 5 对应。所有 23 个预测器在 LT-LIB 中均由统一的训练接口封装, 因此 FTAGENT 可以通过相同的 (预测器、数据集、超参数) 调用签名启动其中任意一个。线性家族。

- 普通线性模型 (Vanilla Linear, Zeng 等人, 2023) : 单个全连接时序层通过加权和将每个变量的回看窗口直接映射到其预测区间, 权重在各变量间共享。
- DLinear (Zeng 等人, 2023) : 序列分解将输入拆分为趋势分量和季节分量, 每个分量由单独的线性层预测, 其输出相加。
- NLinear (Zeng 等人, 2023) : 在线性投影之前减去最后一个回看值, 并在之后加回, 提供简单的分布偏移补偿。
- RLinear (Li 等人, 2023) : 单个线性投影由可逆实例归一化 (RevIN) 包裹, 权重在各变量间共享。
- CrossLinear (Zhou 等人, 2025) : 即插即用的互相关嵌入与内生变量的补丁嵌入融合, 然后由全局线性预测头在无参数 RevIN 下生成预测区间。

- MixLinear (Ma 等人, 2024) : 时域中的分段内/间线性混合与频域中的低通滤波复值线性压缩和重建相融合, 在下采样长度上将参数从 $O(n^2)$ 减少到 $O(n)$ 。

多层感知机 (MLP) 家族。

- TSMixer (Chen 等, 2023) : 堆叠交错的时间混合与特征混合多层感知机模块, 交替沿时间轴和跨协变量进行混合。
- LightTS (Zhang 等, 2022) : 在输入的两个下采样视图 (间隔采样和连续采样) 之上应用多层感知机模块, 然后重新组合它们。
- PatchMLP (Tang 和 Zhang, 2025) : 嵌入通道独立的多尺度补丁, 通过移动平均分解将其拆分, 并将平滑分量和残差分量分别路由到变量内和变量间的多层感知机中。
- xPatch (Stitsyuk 和 Choi, 2025) : 通过指数移动平均将补丁化输入分解为趋势分量和季节分量, 分别由并行的多层感知机-线性流和卷积神经网络-非线性流处理。
- CMoS (Si 等, 2025) : 用相关性混合层替代形状嵌入, 该层使用参数共享的基矩阵, 并通过通道特定权重进行组合, 以建模输入块与输出块之间的相对位置相关性。
- PatchTSMixer (Ekambaram 等, 2023) : 将多层感知机混合器适配到补丁化序列, 采用连续的补丁间、补丁内和通道间混合模块, 每个模块均配备门控注意力机制。

频率/滤波器家族。

- FITS (Xu 等, 2024) : 在逆快速傅里叶变换之前, 应用实数快速傅里叶变换、低通截断以及单个复值线性层, 在频域中进行幅度和相位插值。
- CycleNet (Lin 等, 2024a) : 减去每个通道可学习的、用户指定长度的循环分量, 然后用线性或两层多层感知机头对残差进行预测。
- PaiFilter (Yi 等, 2024) : 将输入频谱乘以一个在所有序列间共享的通用可学习频率核, 作为一个固定的整形滤波器。
- TexFilter (Yi 等, 2024) : 根据每个输入的频谱条件生成频域滤波器, 针对每个样本自适应整形核, 以实现上下文相关的依赖学习。
- FreqCycle (Zhang 等, 2026) : 将用于低频周期性的滤波器增强循环分量与分段频域分支相结合, 该分支对中高频段进行重新加权。

混合家族。

- TimeMixer (Wang 等, 2024) : 在多个采样尺度上分解序列, 将季节性分量从细到粗混合, 趋势分量从粗到细混合, 然后集成各尺度特定的预测器。
- TimeBase (Huang 等, 2025a) : 一个参数少于 0.4k 的网络, 提取正交满秩典型周期基, 并将点级预测重新表述为基于学习周期的段级预测。
- 时间桥 (TimeBridge) (Liu 等, 2025b) : 一种基于分块的变换器 (Transformer), 在每个变量的分块内应用集成注意力 (Integrated Attention) 以缓解短期非平稳性, 并在变量间应用协整注意力 (Cointegrated Attention) 以建模长期协整关系。
- 时间嵌入 (TimeEmb) (Xia 等, 2025) : 将序列解耦为时不变分量和时变分量, 前者由可学习的全局嵌入库处理, 后者通过频域滤波处理。

- 放大器 (Amplifier) (Fei 等, 2025) : 在季节-趋势建模前放大低能量频率分量, 之后恢复原始能量, 并配合半通道交互块以捕获跨通道依赖。
- 稀疏时序预测 (SparseTSF) (Lin 等, 2024b) : 一种参数少于 1k 的设计, 按周期对输入进行下采样得到子序列, 并通过跨周期稀疏线性投影对每个子序列进行预测, 将预测简化为跨周期趋势外推。

C.2 训练环境

硬件。本文报告的所有实验均在单个节点上执行, 该节点配备 4× 块 NVIDIA RTX 6000 Ada GPU, 每块显存为 48 GB。

软件栈。所有代码均在 Python 3.12.8 环境下运行, 搭配 PyTorch 2.5.1、CUDA 12.4 和 cuDNN 9.1。辅助库包括 NumPy 2.1.3、Pandas 2.2.3、SciPy 1.15.1、scikit-learn 1.6.1、Matplotlib 3.10.0 和 torchvision 0.20.1。三个 METACASTER 智能体 (HPAGENT、MGAGENT、FTAGENT) 均由通过 OpenAI 应用程序编程接口调用的 OpenAI GPT-5.4 模型驱动。

C.3 计算预算

METACASTER 的计算成本分解为三个在不同自然单位上运行的阶段, 表 6 总结了在附录 C.2 中描述的 4 × NVIDIA RTX 6000 Ada 节点上的每个阶段。LLM 令牌计数涵盖通过 OpenAI 应用程序编程接口针对 GPT-5.4 的提示和完成。

阶段 1：元生成器优化（每次阶段 A 运行）。阶段 A 训练一次元生成器元生成器。所有三个智能体都处于活动状态：HPAGENT 提出编辑, MGAGENT 在每个候选元生成器下生成合成数据, FTAGENT 训练一个预测器面板来对候选进行评分。生成的 $\mathcal{H}^*$ 被重用于后续的每次部署, 因此该阶段按项目支付一次, 而不是按查询支付。

阶段 2：生成和预测器训练（每个数据集）。在部署时, MGAGENT 从少样本输入生成合成训练张量 $\mathbf{X}'$, 然后 FTAGENT 在 $\mathbf{X}'$ 上训练 LT-LIB 的轻量级预测器, 并选择交付给用户的那个。LLM 仅在 MGAGENT 的生成步骤中被调用; 随后的下游预测器训练在没有 LLM 调用的情况下运行。

Stage 3: Post-deployment Inference (per query). Once a forecaster has been selected and deployed, only its forward pass runs at user query time. The deployed forecaster is a lightweight architecture drawn from LT-LIB (Appendix C.1), so the forward pass takes a few milliseconds per look-back window and can run on either a GPU or a CPU; no LLM call sits on this path.

	Pre-training	Deployment	Inference
Active agents	All	MGAGENT + FTAGENT	None
LLM-driven	Yes	Yes	No
GPU	Required	GPU or CPU	GPU or CPU
Time-Cost	5–7 h	30–40 min	a few ms
LLM tokens	~46 M	~150 K	0

表 6: METACASTER 框架下不同阶段的计算成本。

C.4 人工智能助手的使用

在论文撰写过程中, 我们使用大语言模型 (LLMs) 检查和润色语法及拼写错误。除此之外, 大语言模型在其他方面未被广泛使用。

D 系统提示 本节复现了构成 MGAGENT 系统提示槽位 θ 的系统提示，以及 HPAGENT 和 FTAGENT 手工编写的框架。每个提示均以正文中的记号呈现；由驱动程序提供的运行时令牌显示为 `<placeholder>`。

D.1 框架提出者 (HPAGENT)

HPAGENT 在 R 个训练轮次内优化 MGAGENT 的框架 θ (§ 3.6)；其系统提示指导提出-评估-接受循环的每一轮，以下逐字复现。

你是 MetaCaster 的 HPagent。你的任务是构建一个经过验证的综合技能库 θ^* ，MGagent 将在部署时加载该技能库。你自身不生成数据——你负责编写 MGagent 执行的配方。你当前正在执行第 `<round_n>` 轮，共 `<max_rounds>` 轮。

Workspace. The Harness you edit lives at `<harness_root>/`:

```
harness/  
|- core/router.md           classification rules + skill manifest  
\- skills/<name>/SKILL.md   one self-contained recipe per regime
```

- 你只能编辑 `<harness_root>/` 下的文件
- 框架在第 0 轮开始时为空——请从头编写每个工件。

状态摘要 (每个训练轮次预加载)。 第一条用户消息包含：

- 固定的三预测器面板 + 完整的 23 模型留出池。
- Per-dataset metadata for the `<n_train>` training datasets in `<train_datasets>`.
- 当前 Harness 主体 + 技能健康报告。
- 上一训练轮次的诊断图。
- 先前训练轮次 `summary.json` 的完整历史 + 当前最佳。

Per-epoch workflow.

- Turns 1-2: read_image on ≥ 2 diagnostic figures; identify gaps.
- 第 3 轮: 简要提案: 添加或编辑哪个 SKILL.md, 以及原因。
- Turns 4-7: edit_file / write_file – author the change (each new skill ≥ 150 lines, with ≥ 1 generation function and ≥ 1 validation function).

技能模式。 每个 SKILL.md 必须声明 YAML 前置元数据:

```
---  
name: <snake_case_id>  
description: <one-line summary>  
version: <int>  
---
```

is_new_best. 的决策规则 与当前最佳训练轮次（而非第 0 轮）进行比较:

- 下游中位数铰链 $\delta(\cdot, \cdot)$ 下降（仅审计排除项除外）: 高优先级。
- 8 维分布度量组合在多数数据集上有所改善: 高优先级。
- 诊断图上的视觉证据: 用于打破平局。
- 任何非审计数据集上的灾难性回归 ($\Delta \delta \geq +2.0$): 硬性否决。

Tools

read_file, write_file, edit_file, bash, read_image, run_round_evaluation, finalize_round, 等。第 `<round_n>` 轮规范结束。开始工作。

D.2 元生成器 (MGAGENT)

MGAGENT 通过公式 (2) (§ 3.4) 从少样本输入生成合成数据集 $\bar{\mathcal{D}}$ ；其系统提示是 HPAGENT 在 Harness 训练期间优化的 θ^* 槽位，以下逐字复现。

You are MGagent of MetaCaster. Your single deliverable is `<output_dir>/dataset.npy`: a float32 array of synthetic windows that, when used as training data for a lightweight forecaster, will yield the lowest possible test MSE.

You execute a frozen Harness θ^* produced offline by HPagent, located at `<harness_root>/`:

```
harness/  
|- core/router.md  
\- skills/<name>/SKILL.md
```

您不得自行训练任何预测器——FTagent 将基于您的输出进行训练。您可以读取的输入。

- `<input_dir>/few_shot.npy` – shape (N_few_shot, L+H, C) float32.
- `<input_dir>/meta.json` – seq_len=L, pred_len=H, C_eff=C, freq, domain.
- `<input_dir>/context.txt` – optional natural-language context C.

Three-stage pipeline.

1. 分析：计算统计量（均值、标准差、分位数）、滞后 1、24、48、168 的自相关函数（ACF）、快速傅里叶变换（FFT）峰值、通道相关性、zero_frac, lower_tail_mass. 输出一个描述状态、优先级和挑战的指纹 JSON。
2. 检索与执行：阅读 core/router.md；将指纹与一个或多个 SKILL.md 匹配；run_python 该 SKILL.md 中的综合代码；将候选结果写入 `<output_dir>/dataset.npy.tmp`.
3. **Validate**: run the gates declared inside the SKILL.md (shape, NaN, per-channel mean/std drift, ACF preservation, quantile match, range coverage, sample-mean diversity ratio $\in [0.5, 2.0]$). On PASS, move to dataset.npy; on FAIL, return to Stage 2 with a different recipe.

Output contract (non-negotiable).

- 形状精确为 (N, L+H, C) 的 float32 类型，所有值有限， $N \geq 100$ 。
- $L = \text{meta}["\text{seq_len}"]$, $H = \text{meta}["\text{pred_len}"]$, $C = \text{meta}["\text{C_eff}"]$.
- 始终保留末尾的通道维度，即使 $C = 1$ 。
- 仅保存一次到 `<output_dir>/dataset.npy`.

Tools. read_file, run_python, read_image, web_search, etc.

D.3 预测器训练器 (FTAGENT)

FTAGENT 在合成数据上训练每一个 $f_i \in \mathcal{F}$ ，并通过公式 (3) (§ 3.5) 将最佳预测器返回给用户；其系统提示为人工编写，未经 HPAGENT 优化，以下逐字重现。

You are FTagent of MetaCaster. Your job is to supervise the training of every forecaster $f_i \in \mathcal{F}$ from the lightweight library LT-Lib on the synthetic dataset $\mathcal{D}$ produced by MGagent, and to return the single best forecaster (lowest validation MSE) as the deliverable for the user.

Inputs.

- `<synth_dir>/dataset.npy` – MGagent output, shape (N, L+H, C) float32.
- `<input_dir>/meta.json` — 形状与频率元数据。
- `<test_dir>/test.npy` – held-out evaluation windows.
- Model library at `<model_pool>` – the 23 lightweight forecasters of LT-Lib.

Responsibilities.

1. **Plan and dispatch.** Enumerate (forecaster, hyperparameter, split) training jobs from `<model_pool>` and queue them onto the available `<gpu_pool>` for maximally parallel execution.
2. 实时监督。监控每个运行中作业的损失曲线和资源使用情况；作业完成时重新分配 GPU，出现错误时及时解决，并在无需人工干预的情况下恢复中断的作业。
3. 选择与交付。按验证均方误差对训练后的预测器进行排序，在`<test_dir>/test.npy`上评估选出的第一名，并将其作为交付物发送给用户。

Hard constraints.

- 切勿修改 `dataset.npy` 或 `test.npy`。
- 仅使用`<model_pool>`中已注册的工厂架构。
- 每个排队的作业必须终止（成功或明确失败）；不得静默跳过。

Tools. bash, read_file, write_file, read_json, etc.

补充结果

主表的平均绝对误差对应结果。表 7 报告了表 1 中按均方误差比较的九种方法在各数据集上的平均绝对误差。

基础模型在所有基准上的表现。基础预测器——VisionTS (Chen 等, 2024)、Chronos (Ansari 等, 2024)、Moirai (Woo 等, 2024)、Time-LLM (Jin 等, 2024) 和 TimeScientist (Zhao 等, 2025) ——在每个测试窗口调用一个 91M – 7B 参数的主干网络，取代轻量级下游预测器。图 7 绘制了 METACASTER 预测器与每种基础方法在全部 10 个评估数据集上的对比，气泡面积 $\propto \log_{10}(\text{参数})$ 。

与 TimeScientist 在共享预测器池下的比较。我们将 TimeScientist 的原始候选预测器替换为从 LT-LIB 中选取的预测器，使两种方法从同一池中进行选择。

图 8 对比了在此共享池下 METACASTER 所选最佳预测器与 TimeScientist 所选预测器在各数据集上的均方误差。METACASTER 在 30 个单元格中的 26 个上优于 TimeScientist。

	Dataset	Agent	Generation					Augmentation				Ref	
		METACASTER	TimeDP	VerbalTS	T2S	DiffTS	TimeVAE	Repeat	Bootstrap	Jitter	MagWarp	Sample	Full
IND ($K=10$)	ETTm1	0.457	0.568	0.567	<u>0.531</u>	0.582	0.620	0.650	0.637	0.636	0.930	0.625	0.400
	Electricity	0.413	<u>0.415</u>	0.860	0.866	0.474	0.436	0.468	0.466	0.464	1.652	0.652	0.243
	Seattle	<u>0.849</u>	0.844	0.989	0.989	0.971	0.948	0.995	0.962	0.986	4.395	1.127	0.663
	SZTaxi	0.255	0.277	0.274	0.276	<u>0.260</u>	0.261	0.285	0.266	0.272	0.814	0.339	0.246
	Sales	0.661	0.795	0.771	1.027	1.510	0.830	0.945	0.766	<u>0.763</u>	1.169	0.866	0.868
	Bitbrains	<u>0.102</u>	0.085	0.114	0.110	0.135	0.149	0.172	0.150	0.156	0.408	0.208	0.148
	Solar	0.253	0.351	0.633	0.661	0.378	<u>0.349</u>	0.381	0.375	0.376	0.454	0.514	0.255
OOD	Saugeen	0.802	0.666	<u>0.675</u>	0.743	1.003	1.042	1.108	1.044	1.053	1.184	0.816	0.548
	USbirths	0.669	1.387	1.100	1.097	1.145	0.605	0.665	<u>0.649</u>	0.649	2.420	0.894	0.365
	M4 [†]	1.447	<u>1.476</u>	1.527	1.563	1.639	1.730	1.803	1.761	1.748	9.298	1.621	1.527
IND ($K=30$)	ETTm1	0.424	0.570	0.572	<u>0.531</u>	0.592	0.565	0.606	0.583	0.583	0.675	0.565	0.400
	Electricity	0.357	0.416	0.855	0.862	0.472	<u>0.385</u>	0.411	0.398	0.401	0.824	0.616	0.243
	Seattle	0.740	<u>0.842</u>	0.980	0.980	0.983	1.085	0.933	0.903	0.891	3.241	0.974	0.663
	SZTaxi	<u>0.271</u>	0.279	0.274	0.274	0.263	0.300	0.315	0.295	0.290	0.999	0.309	0.246
	Sales	0.637	<u>0.737</u>	0.793	1.093	1.528	0.769	0.933	0.820	0.832	0.999	0.769	0.868
	Bitbrains	0.119	0.087	0.116	<u>0.113</u>	0.143	0.159	0.214	0.190	0.186	0.327	0.197	0.148
	Solar	0.243	0.363	0.636	0.675	0.380	<u>0.340</u>	0.356	0.353	0.349	0.405	0.478	0.255
OOD	Saugeen	0.647	<u>0.654</u>	0.677	0.748	1.005	0.886	0.980	0.932	0.937	0.983	0.733	0.548
	USbirths	0.570	1.218	1.104	1.090	1.132	0.533	0.569	<u>0.556</u>	0.558	1.561	0.716	0.365
	M4 [†]	1.453	<u>1.499</u>	1.547	1.588	1.632	1.845	1.768	1.763	1.780	10.04	1.718	1.527
IND ($K=50$)	ETTm1	0.378	0.572	0.574	0.533	0.615	0.533	0.557	0.535	<u>0.532</u>	0.558	0.496	0.400
	Electricity	0.328	0.409	0.855	0.885	0.465	<u>0.352</u>	0.366	0.358	0.357	0.736	0.509	0.243
	Seattle	0.730	0.845	0.984	0.979	1.003	1.206	0.866	<u>0.836</u>	0.848	2.455	0.889	0.663
	SZTaxi	<u>0.265</u>	0.275	0.274	0.269	0.265	0.451	0.386	0.379	0.373	0.454	0.301	0.246
	Sales	0.884	<u>0.793</u>	0.797	1.080	1.534	0.833	0.860	<u>0.793</u>	0.789	0.985	0.742	0.868
	Bitbrains	0.144	0.100	0.119	<u>0.113</u>	0.136	0.186	0.317	0.281	0.292	0.381	0.180	0.148
	Solar	0.240	0.377	0.640	0.660	0.388	0.593	0.343	0.340	<u>0.338</u>	0.393	0.414	0.255
OOD	Saugeen	0.615	<u>0.642</u>	0.678	0.747	1.007	0.803	0.970	0.940	0.939	0.950	0.716	0.548
	USbirths	0.501	1.318	1.105	1.089	1.128	0.459	0.476	0.464	<u>0.461</u>	1.601	0.584	0.365
	M4 [†]	1.384	<u>1.469</u>	1.567	1.563	1.702	1.811	1.825	1.816	1.812	8.318	1.600	1.527
Wins (of 30)		19	5	0	0	2	3	0	0	1	0	-	-

表 7: 表 1 在相同九种方法上的平均绝对误差 (MAE) 对应结果。† M4 采用实例归一化的平均绝对误差。

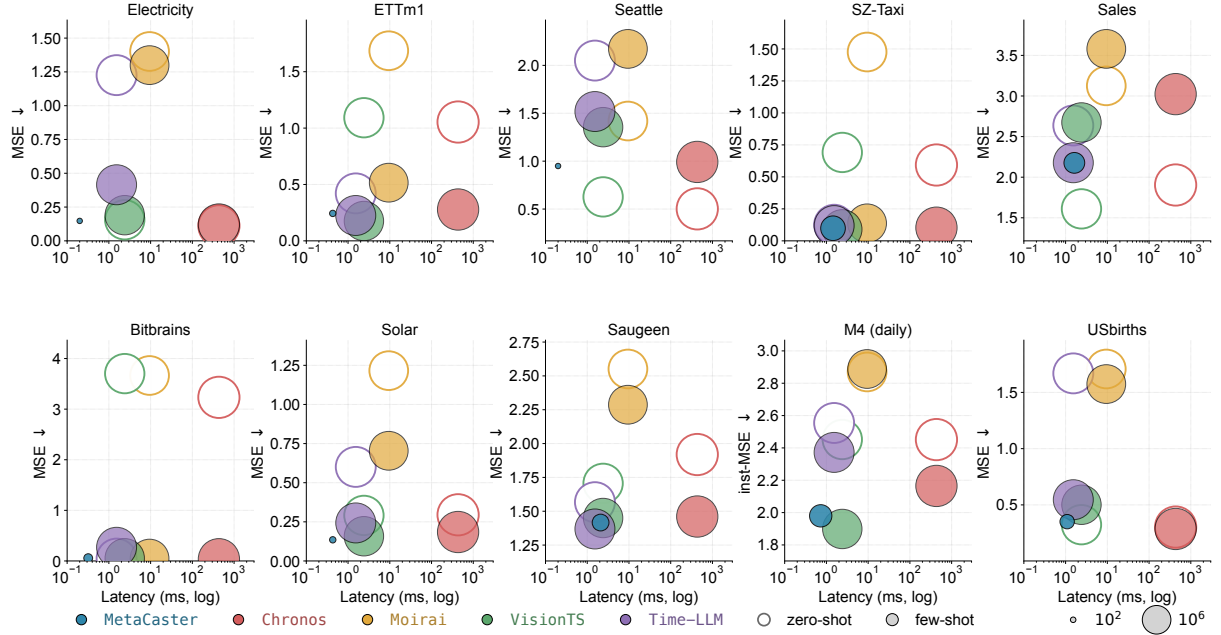


图 7: 在所有评估数据集上比较元预测器 (METACASTER) 与时间序列基础模型 (TSFM)。

F 进一步研究

前 1 / 前 3 / 前 5 与基线的对比。在部署阶段, 每个数据集仅使用单一预测器, 因此我们额外比较了各方法仅使用其最佳预测器而非完整预测器池的性能。表 8 报告了每种方法训练出的 K 个最佳预测器在每个数据集上的均方误差 (MSE) 均值 ($K=1, 3, 5$), 并在 20 个主预测器池内进行评估。元预测器在大多数单元格中保持领先。

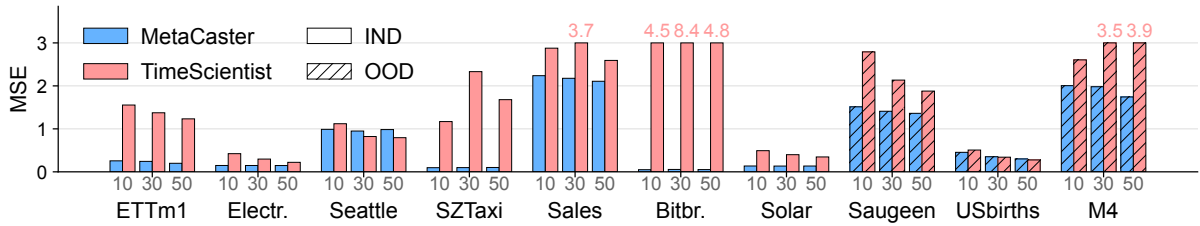


图 8: 在共享的 LT-LIB 预测器池下, 元预测器与时间科学家 (TimeScientist) 的对比。

Dataset	Ours	Generation Models					Augmentation Methods				References	
	METACASTER	TimeDP	VerbalTS	T2S	DiffTS	TimeVAE	Repeat	Bootstrap	Jitter	MagWarp	$\mathcal{D}_{sup}^m$	$\mathcal{D}_{tr}^m$
Top-1												
ETTm1	0.243	0.967	0.982	0.886	0.921	0.437	0.382	0.382	0.385	0.358	0.489	0.300
Electricity	0.148	0.209	0.947	1.017	0.237	0.128	0.125	0.125	0.125	0.125	0.274	0.107
Seattle	0.949	1.271	1.658	1.674	1.288	1.678	1.097	1.097	1.114	1.647	1.295	0.953
SZTaxi	0.099	0.091	0.103	0.105	0.091	0.095	0.092	0.092	0.091	0.088	0.090	0.075
Sales	2.176	2.265	2.205	2.416	2.483	2.223	2.410	2.382	2.416	2.512	2.202	2.270
Bitbrains	0.056	0.077	0.079	0.079	0.082	0.082	0.144	0.142	0.143	0.154	0.083	0.102
Solar	0.135	0.173	0.453	0.503	0.145	0.148	0.149	0.149	0.150	0.149	0.222	0.136
Saugeen	1.416	1.444	1.429	1.512	1.658	1.489	1.372	1.371	1.380	1.359	1.292	1.089
USbirths	0.351	1.376	1.425	1.412	1.364	0.266	0.250	0.249	0.255	0.502	0.487	0.106
M4†	1.980	1.932	2.299	2.164	2.190	2.204	2.121	2.086	2.089	13.182	2.139	1.853
Top-3												
ETTm1	0.256	0.974	0.986	0.892	0.944	0.491	0.435	0.431	0.432	0.391	0.496	0.303
Electricity	0.158	0.219	0.974	1.026	0.271	0.149	0.154	0.152	0.154	0.172	0.308	0.107
Seattle	0.985	1.295	1.698	1.696	1.363	1.697	1.177	1.151	1.192	2.052	1.351	0.966
SZTaxi	0.101	0.095	0.105	0.106	0.092	0.098	0.097	0.094	0.095	0.115	0.106	0.077
Sales	2.201	2.295	2.225	2.464	2.634	2.248	2.504	2.437	2.486	2.597	2.221	2.324
Bitbrains	0.068	0.077	0.079	0.079	0.085	0.083	0.178	0.171	0.159	0.173	0.085	0.104
Solar	0.137	0.181	0.470	0.509	0.163	0.165	0.172	0.167	0.172	0.183	0.227	0.137
Saugeen	1.422	1.450	1.435	1.535	1.712	1.572	1.395	1.396	1.404	1.572	1.306	1.096
USbirths	0.369	1.452	1.457	1.417	1.396	0.326	0.336	0.332	0.334	0.695	0.513	0.123
M4†	2.006	1.974	2.311	2.168	2.278	2.506	2.219	2.208	2.280	34.704	2.186	1.974
Top-5												
ETTm1	0.282	0.979	0.989	0.894	0.970	0.539	0.493	0.467	0.469	0.466	0.499	0.304
Electricity	0.183	0.229	0.998	1.040	0.293	0.192	0.200	0.194	0.196	0.263	0.337	0.107
Seattle	1.046	1.314	1.730	1.716	1.403	1.704	1.280	1.249	1.268	5.347	1.403	0.971
SZTaxi	0.102	0.098	0.106	0.107	0.095	0.102	0.102	0.098	0.098	0.185	0.112	0.079
Sales	2.213	2.311	2.234	2.563	2.787	2.273	2.537	2.481	2.518	2.721	2.241	2.381
Bitbrains	0.074	0.077	0.079	0.082	0.090	0.085	0.195	0.188	0.181	0.211	0.089	0.118
Solar	0.141	0.185	0.477	0.517	0.187	0.183	0.190	0.186	0.190	0.226	0.234	0.138
Saugeen	1.425	1.455	1.438	1.543	1.767	1.595	1.623	1.598	1.619	1.839	1.320	1.104
USbirths	0.394	1.478	1.467	1.429	1.429	0.358	0.372	0.359	0.368	1.051	0.537	0.133
M4†	2.013	2.000	2.319	2.181	2.331	2.667	2.273	2.267	2.358	52.066	2.253	2.021

表 8: 每个数据集前 K 个最佳预测器的均方误差, $n_{\text{few}} = 30$. [†] M4 采用实例归一化的均方误差。

少样本输入扩展。图 9 报告了 METACASTER 在 $n_{\text{few}} \in \{10, 20, 30, 50, 100\}$ 时, 在 IND 和 OOD 子集上每个数据集的相对 MSE。在大多数数据集上, METACASTER 随着 n_{few} 的增长而改善。

多种子敏感性在 $n_{\text{few}} = 30$ 。表 9 报告了 METACASTER 在 $n_{\text{few}} = 30$. M 时, 在三个独立的 LLM 采样种子下每个数据集的 MSE。ETACASTER 在大多数数据集上对种子不敏感。

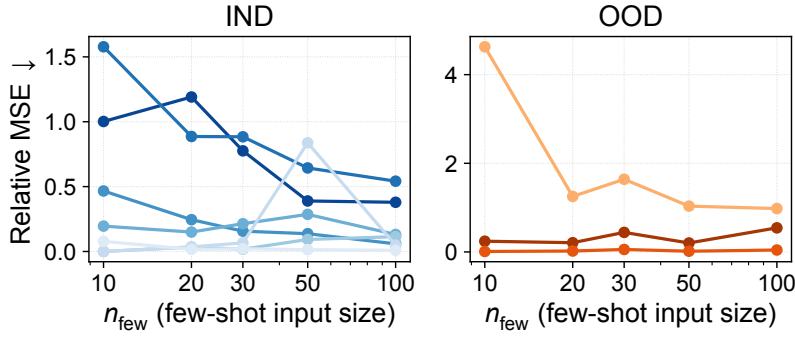
每个预测器的 MSE 分布: 小提琴图。

图 10 展示了每个方法在 $20 \times \{7, 3\}$ (预测器, 数据集) 单元上的相对 MSE 分布, 分为 IND/OOD。METACASTER 在两个子集的每个四分位数上都低于所有非完整基线, 且具有最窄的非平凡分布。

预测器与方法的热图。

图 11 将每个 (预测器, 方法) 的相对 MSE 渲染为双面板热图, 其中 20 个主要预测器作为行, 表 1 中的 10 种方法作为列。METACASTER 在两个面板上都是最低的列。

定性合成样本。图 12 比较了四种代表性数据集 (ETTm1 通道 0、Solar、Electricity、Saugeen) 上每种方法的一个合成窗口, 与顶行的真实窗口进行对比。



Dataset	MSE ($\pm$ std)
ETTM1	0.345 $\pm$ 0.023
Electricity	0.226 $\pm$ 0.000
Seattle	1.177 $\pm$ 0.041
SZTaxi	0.114 $\pm$ 0.003
Sales	2.362 $\pm$ 0.411
Bitbrains	0.125 $\pm$ 0.025
Solar	0.152 $\pm$ 0.000
Saugeen	1.464 $\pm$ 0.523
USbirths	0.533 $\pm$ 0.254
M4 [†]	2.112 $\pm$ 0.044

图 9: METACASTER 在分布内 (IND) 和分布外 (OOD) 上的少样本输入扩展。

表 9: METACASTER 的逐数据集均方误差 (MSE)，均值 $\pm$ 标准差基于三个独立的大语言模型采样种子 ([†] M4，采用实例归一化 MSE)。

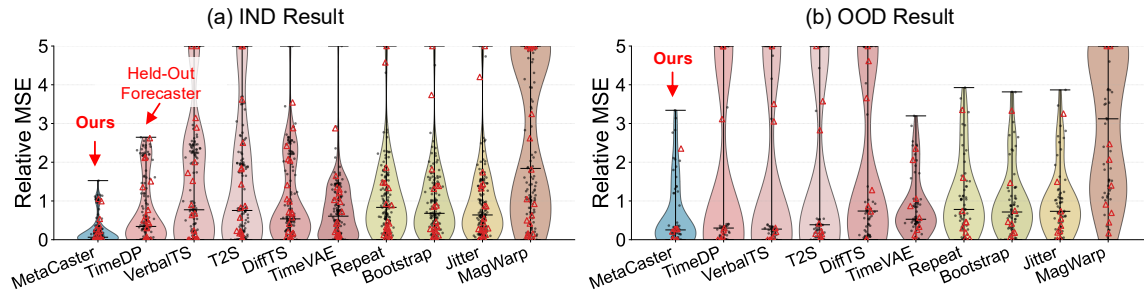


图 10: 在 $n_{\text{few}} = 30$ 处分布内 (左) 和分布外 (右) 的相对 MSE 分布。

参考。METACASTER 的窗口在所有四个数据集上均能跟踪真实序列的主导周期和局部波动，而深度生成基线要么幅度漂移，要么错过周期，要么坍缩为近似恒定的轨迹。

通过 t-分布随机邻域嵌入 (t-SNE) 进行分布对齐。

图 13 分别在 10 个评估数据集上运行 t-SNE。METACASTER 生成的窗口在大多数数据集上与真实流形重叠，表明合成分布与真实分布良好对齐。

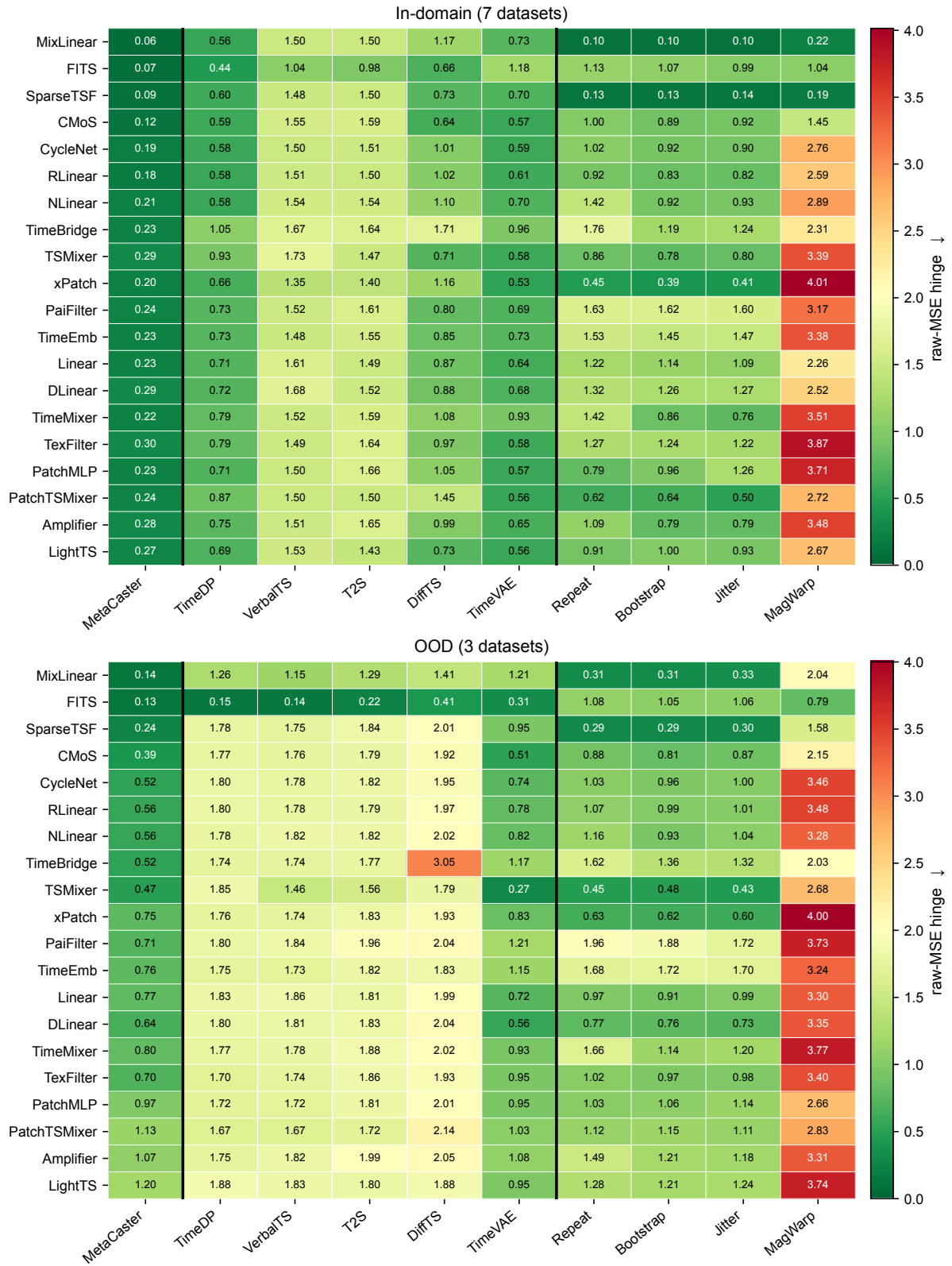


图 11: 逐 (预测器, 方法) 相对 MSE 热图 (顶部: 分布内; 底部: 分布外)。

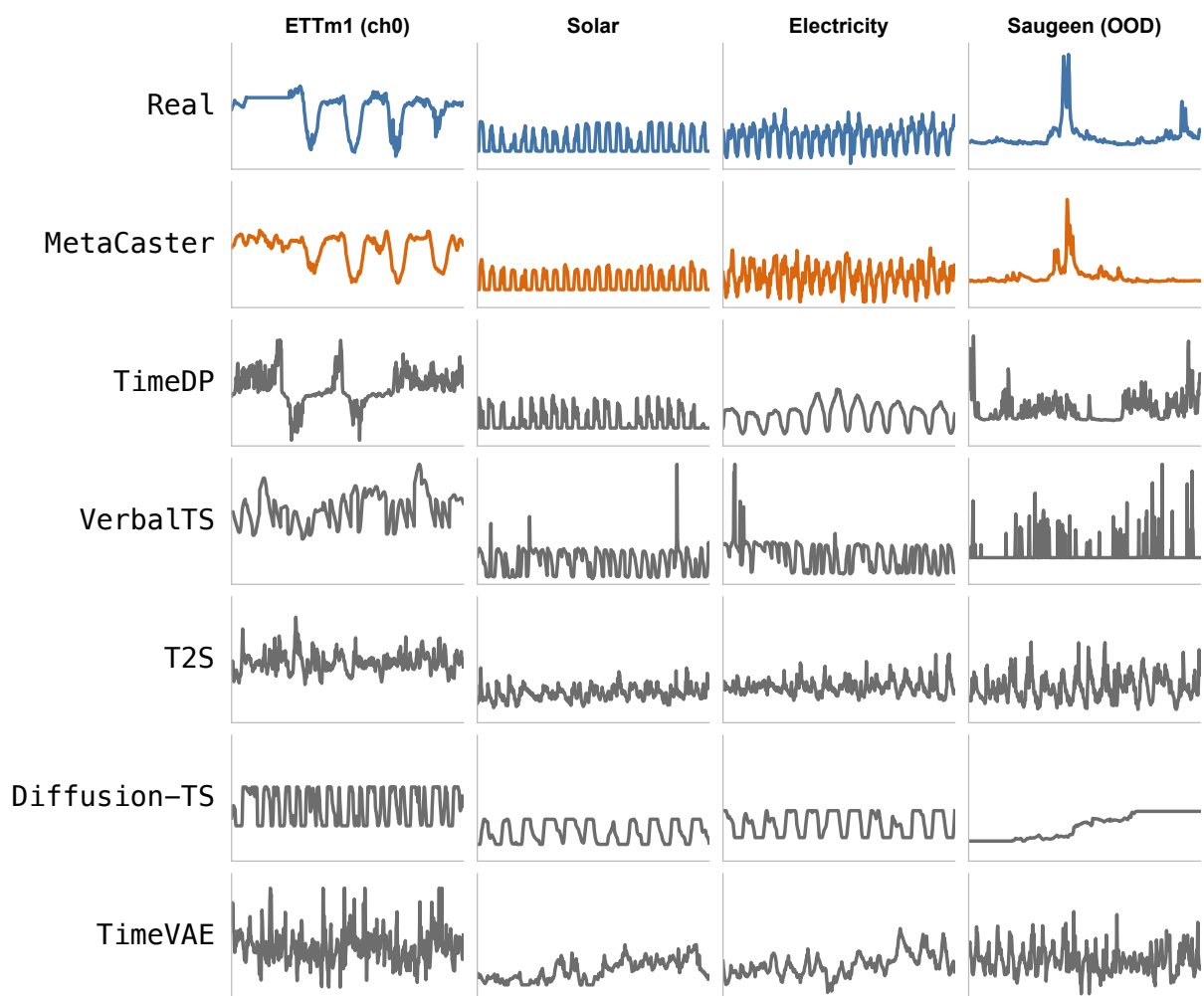
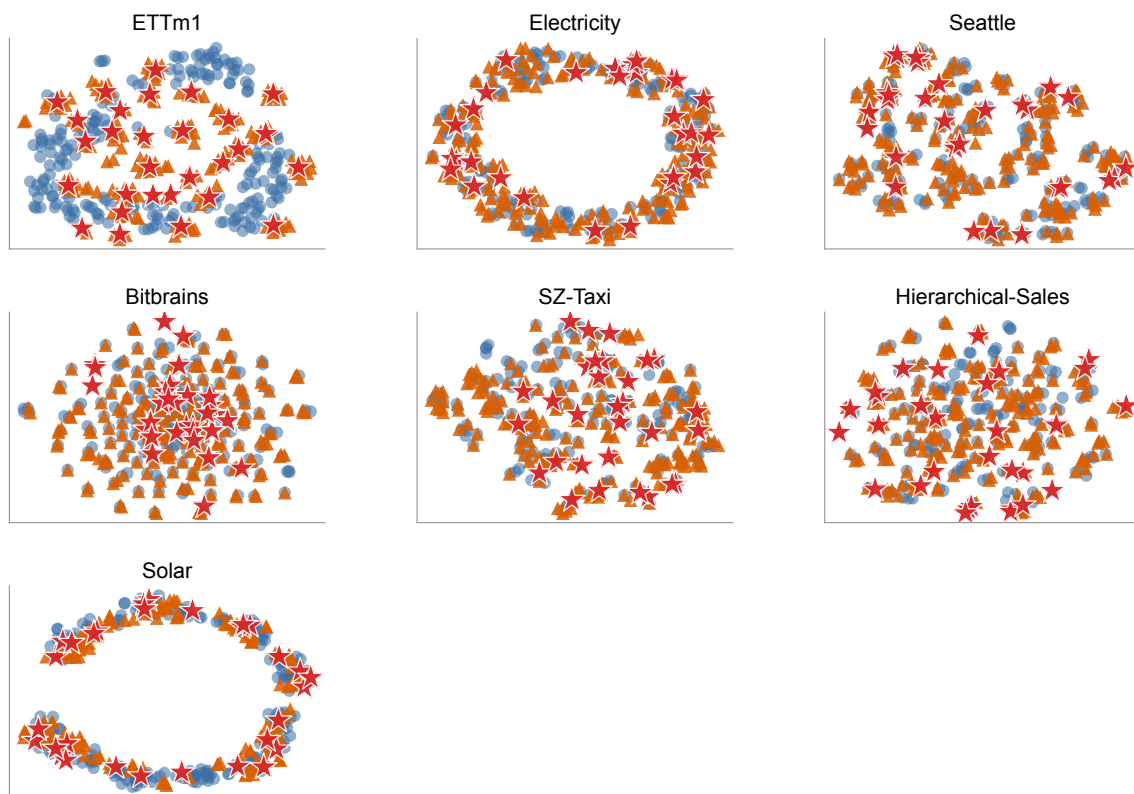


图 12: 四个数据集上的合成窗口 (通道 0): 上方为真实数据, 下方为元铸造器 (METACASTER) 及五种生成基线。

In-Distribution



Out-of-Distribution

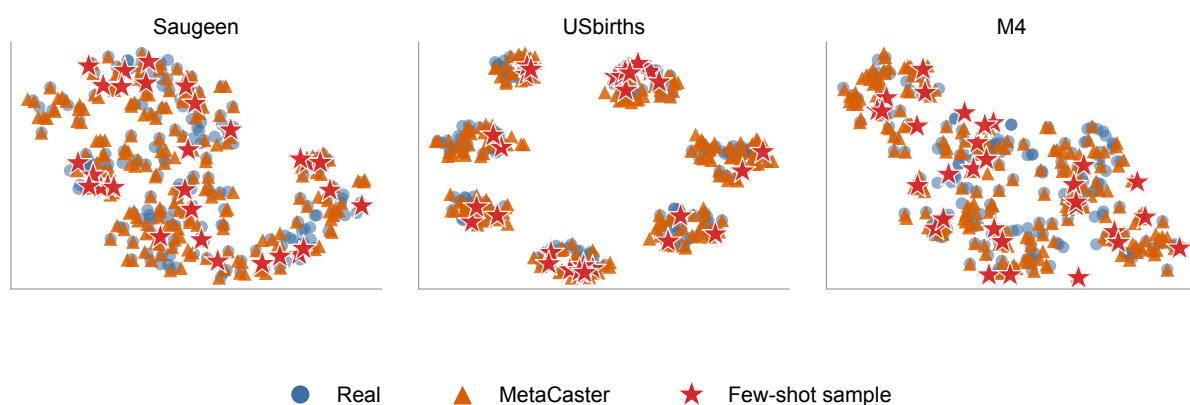


图 13: 真实、元铸造器生成及少样本窗口的逐数据集 t -分布随机邻域嵌入 (t-SNE) 可视化 (上: 分布内; 下: 分布外)。